

HƯỚNG DẪN VỀ NỀN TẢNG TRI THỨC KỸ THUẬT PHẦN MỀM

*Guide to the Software Engineering
Body of Knowledge*

Biên soạn tham khảo bằng tiếng Việt



MỤC LỤC

Lời nói đầu

Chương 1. Nền tảng Yêu cầu Phần mềm (*Software Requirements*)

Chương 2. Kiến trúc Phần mềm (*Software Architecture*)

Chương 3. Thiết kế Phần mềm (*Software Design*)

Chương 4. Xây dựng Phần mềm (*Software Construction*)

Chương 5. Nền tảng Kiểm thử Phần mềm (*Software Testing*)

Chương 6. Vận hành KỸ THUẬT PHẦN MỀM (*Software Engineering Operations*)

Chương 7. Bảo trì Phần mềm (*Software Maintenance*)

Chương 8. Quản lý Cấu hình Phần mềm (*Software Configuration Management*)

Chương 9. Quản lý KỸ THUẬT PHẦN MỀM (*Software Engineering Management*)

Chương 10. Quy trình KỸ THUẬT PHẦN MỀM (*Software Engineering Process*)

Chương 11. Mô hình và Phương pháp KỸ THUẬT PHẦN MỀM
(*Software Engineering Models and Methods*)

Chương 12. Chất lượng Phần mềm (*Software Quality*)

Chương 13. An toàn Phần mềm (*Software Security*)

Chương 14. Thực hành Nghề nghiệp KỸ THUẬT PHẦN MỀM
(*Software Engineering Professional Practice*)

Chương 15. Kinh tế học KỸ THUẬT PHẦN MỀM (*Software Engineering Economics*)

Chương 16. Nền tảng Điện toán (*Computing Foundations*)

Chương 17. Nền tảng Toán học (*Mathematical Foundations*)

Chương 18. Nền tảng Kỹ Thuật (*Engineering Foundations*)

CHƯƠNG 1

Nền tảng Yêu cầu Phần mềm

Software Requirements

Yêu cầu phần mềm mô tả những gì hệ thống phần mềm phải làm và những ràng buộc mà nó phải tuân thủ. Đây là lĩnh vực tri thức nền tảng, quyết định phần lớn thành công hay thất bại của một dự án phần mềm, bởi vì phần lớn lỗi nghiêm trọng và chi phí sửa lỗi phát sinh từ việc hiểu sai hoặc đặc tả sai yêu cầu ngay từ giai đoạn đầu.

Góc nhìn chuyên sâu: một trong những sai lầm phổ biến nhất của các đội phát triển non kinh nghiệm là nhầm lẫn giữa "thu thập yêu cầu" và "thu thập giải pháp" - khi được hỏi khách hàng thường mô tả ngay một giải pháp cụ thể ("tôi muốn có nút bấm màu đỏ ở góc trên") thay vì nhu cầu thực sự đằng sau ("tôi cần thao tác này được thực hiện nhanh và không thể bỏ sót"). Kỹ sư yêu cầu giỏi phải liên tục hỏi "tại sao" để lần ngược từ giải pháp đề xuất về nhu cầu gốc, vì bám chặt vào giải pháp do khách hàng đề xuất quá sớm sẽ giới hạn không gian thiết kế và có thể bỏ lỡ giải pháp tối ưu hơn. Trong bối cảnh 2026, xu hướng "Requirements-as-Code" (yêu cầu dưới dạng mã có thể kiểm tra tự động, như minh họa ở Ví dụ 1.2) đang giúp thu hẹp khoảng cách giữa đặc tả và triển khai, nhưng không thay thế được kỹ năng phỏng vấn và thấu hiểu nghiệp vụ của con người.

1. Nền tảng Yêu cầu Phần mềm

Yêu cầu là một khả năng hoặc điều kiện mà hệ thống phải có để thỏa mãn nhu cầu của các bên liên quan (stakeholders). Yêu cầu được phân loại thành yêu cầu chức năng (functional requirements mô tả hành vi mà hệ thống phải thực hiện), yêu cầu phi chức năng (non-functional requirements các thuộc tính chất lượng như hiệu năng, độ tin cậy, khả năng bảo trì, an toàn) và các ràng buộc thiết kế hoặc triển khai. Kỹ nghệ yêu cầu (requirements engineering) là toàn bộ tập hợp các hoạt động dùng để khám phá, phân tích, đặc tả và quản lý yêu cầu trong suốt vòng đời dự án.

2. Thu thập Yêu cầu (Requirements Elicitation)

Là quá trình tìm hiểu nhu cầu thực sự của người dùng và các bên liên quan thông qua các kỹ thuật như phỏng vấn, khảo sát, hội thảo (workshop), quan sát tại hiện trường, phân tích tài liệu nghiệp vụ, xây dựng kịch bản sử dụng (use case, user story) và tạo mẫu thử nghiệm (prototyping). Thách thức lớn nhất của giai đoạn này là các bên liên quan thường không thể diễn đạt đầy đủ, chính xác hoặc nhất quán về nhu cầu của họ, đòi hỏi kỹ sư yêu cầu phải chủ động khai thác tri thức tiềm ẩn.

3. Phân tích Yêu cầu (Requirements Analysis)

Sau khi thu thập, các yêu cầu thô cần được phân loại, mô hình hóa, cấu trúc hóa và tinh chỉnh nhằm phát hiện mâu thuẫn, trùng lặp, thiếu sót hoặc mơ hồ. Các kỹ thuật phổ biến bao gồm mô hình hóa hướng đối tượng (UML), mô hình hóa dữ liệu, phân tích kịch bản và phân loại yêu cầu theo mức độ ưu tiên (prioritization) để chuẩn bị cho đặc tả.

4. Đặc tả Yêu cầu (Requirements Specification)

Là hoạt động ghi lại yêu cầu dưới dạng tài liệu có cấu trúc, rõ ràng và có thể kiểm chứng được - ví dụ tài liệu Đặc tả Yêu cầu Phần mềm (Software Requirements Specification SRS). Đặc tả tốt cần đạt các thuộc tính: đầy đủ, nhất quán, không mơ hồ, có thể xác minh, có thể truy vết (traceable) và được sắp xếp theo mức ưu tiên.

5. Xác nhận Yêu cầu (Requirements Validation)

Xác nhận nhằm bảo đảm rằng tập yêu cầu đã đặc tả thực sự phản ánh đúng nhu cầu của các bên liên quan, thông qua các kỹ thuật như rà soát (review), kiểm tra thử (walkthrough), duyệt hình thức (formal inspection) và xây dựng nguyên mẫu để người dùng phản hồi sớm.

6. Hoạt động Quản lý Yêu cầu

Quản lý yêu cầu bao gồm việc kiểm soát thay đổi yêu cầu, quản lý phiên bản, thiết lập ma trận truy vết (traceability matrix) liên kết yêu cầu với thiết kế, mã nguồn và test case, cũng như quản lý mối quan hệ và mức độ ưu tiên giữa các yêu cầu trong suốt vòng đời sản phẩm.

7. Xem xét Thực tiễn

Trong thực tế, kỹ nghệ yêu cầu phải thích ứng với bối cảnh cụ thể: dự án Agile thường dùng backlog và user story thay cho SRS đầy đủ; dự án an toàn nghiêm ngặt (safety-critical) đòi hỏi đặc tả hình thức chặt chẽ hơn; và các yếu tố văn hóa, tổ chức, quy mô nhóm đều ảnh hưởng đến cách tiếp cận yêu cầu.

8. Công cụ Yêu cầu Phần mềm

Các công cụ hỗ trợ bao gồm phần mềm quản lý yêu cầu chuyên dụng (như Jama, DOORS), công cụ quản lý backlog Agile (Jira, Azure DevOps), công cụ mô hình hóa UML, và công cụ truy vết tự động giúp liên kết yêu cầu với các sản phẩm công việc khác.

Minh họa và Ví dụ Thực tế



Hình 1.1 - Quy trình kỹ nghệ yêu cầu với vòng lặp phản hồi liên tục.

Gherkin - Đặc tả yêu cầu dạng BDD (2026: dùng trực tiếp làm test tự động)

Ví dụ 1.1 - Đặc tả yêu cầu theo phong cách Behavior-Driven Development (BDD), phổ biến trong các nhóm Agile hiện đại vì đặc tả có thể chạy trực tiếp như kịch bản kiểm thử chấp nhận (dùng công cụ như Cucumber, Behave, SpecFlow):

Feature: Thanh toán đơn hàng bằng ví điện tử

Là một khách hàng, tôi muốn thanh toán bằng ví điện tử để hoàn tất đơn hàng nhanh chóng và an toàn.

Scenario: Thanh toán thành công với số dư đủ

Given khách hàng đã đăng nhập và có đơn hàng trị giá 250,000 VND

And số dư ví điện tử của khách hàng là 500,000 VND

When khách hàng chọn phương thức "Ví điện tử" và xác nhận thanh toán

Then hệ thống trừ 250,000 VND từ ví điện tử

And đơn hàng chuyển sang trạng thái "Đã thanh toán"

And khách hàng nhận được thông báo xác nhận trong vòng 3 giây

Scenario: Từ chối thanh toán khi số dư không đủ

Given khách hàng có đơn hàng trị giá 250,000 VND

And số dư ví điện tử của khách hàng là 100,000 VND

When khách hàng chọn phương thức "Ví điện tử" và xác nhận thanh toán

Then hệ thống hiển thị lỗi "Số dư không đủ"

And đơn hàng vẫn ở trạng thái "Chờ thanh toán"

Python - Xác thực và truy vết yêu cầu bằng Pydantic (Requirements-as-Code)

Ví dụ 1.2 - Thực hành hiện đại: mô hình hóa yêu cầu như dữ liệu có cấu trúc (structured data) thay vì văn bản tự do, cho phép kiểm tra tự động tính đầy đủ và khả năng truy vết (traceability) ngay trong pipeline CI:

```
from pydantic import BaseModel, field_validator
from enum import Enum

class Priority(str, Enum):
    MUST = "must-have"
    SHOULD = "should-have"
    COULD = "could-have"

class Requirement(BaseModel):
    id: str          # ví dụ: 'REQ-042'
    description: str
    priority: Priority
    linked_test_cases: list[str] = []

    @field_validator("linked_test_cases")
    @classmethod
    def must_have_least_one_test(cls, v, info):
        # Ràng buộc: yêu cầu MUST-HAVE bắt buộc có test case truy vết
        if info.data.get("priority") == Priority.MUST and not v:
            raise ValueError(
                f"Yêu cầu {info.data.get('id')} là MUST-HAVE nhưng chưa có test case"
            )
        return v

# Chạy trong CI để chặn merge nếu yêu cầu quan trọng thiếu truy vết kiểm thử
requirement = Requirement(
    id="REQ-042",
    description="Hệ thống phải hỗ trợ thanh toán qua ví điện tử",
    priority=Priority.MUST,
    linked_test_cases=["TC-201", "TC-202"],
)
```

Bài tập Thực hành

Bài 1. Chọn một chức năng bất kỳ trong một ứng dụng bạn hay dùng (ví dụ: đặt xe, đặt đồ ăn). Viết lại yêu cầu của chức năng đó theo định dạng Gherkin (Given–When–Then) với ít nhất 2 kịch bản, bao gồm 1 kịch bản lỗi (negative case).

Bài 2. Xây dựng một bảng ma trận truy vết (traceability matrix) đơn giản gồm 5 dòng, liên kết: Mã yêu cầu → Mã thiết kế liên quan → Mã test case liên quan → Trạng thái (Chưa làm/Đang làm/Hoàn thành).

Bài 3. Thảo luận: với dự án Agile dùng backlog thay cho SRS đầy đủ, làm thế nào để vẫn đảm bảo tính "xác nhận được" (validatable) của yêu cầu? Liệt kê 3 rủi ro có thể xảy ra nếu bỏ qua bước Requirements Validation.

CHƯƠNG 2

Kiến trúc Phần mềm

Software Architecture

Đây là một trong ba lĩnh vực tri thức hoàn toàn mới được bổ sung trong SWEBOK V4.0, phản ánh vai trò ngày càng quan trọng của kiến trúc phần mềm như cầu nối giữa yêu cầu và thiết kế chi tiết, quyết định cấu trúc tổng thể, khả năng mở rộng và tiến hóa lâu dài của hệ thống.

Góc nhìn chuyên sâu: kiến trúc microservices không phải lúc nào cũng là lựa chọn đúng - đây là một trong những ngộ nhận phổ biến nhất trong ngành. Chia nhỏ hệ thống thành nhiều dịch vụ độc lập giúp mở rộng đội ngũ và scale từng phần riêng biệt, nhưng đổi lại là độ phức tạp vận hành tăng vọt: cần xử lý giao dịch phân tán (distributed transactions), độ trễ mạng giữa các service, và gỡ lỗi khó khăn hơn nhiều so với một hệ thống nguyên khối (monolith). Nhiều công ty công nghệ lớn năm 2024-2026 đã công khai chia sẻ kinh nghiệm "quay lại monolith" (hoặc áp dụng "modular monolith") sau khi nhận ra chi phí vận hành microservices vượt quá lợi ích với quy mô đội ngũ của họ. Quyết định kiến trúc luôn phải cân nhắc dựa trên quy mô đội ngũ, tốc độ tăng trưởng dự kiến và năng lực vận hành hiện có - không nên chọn theo trào lưu.

1. Nền tảng Kiến trúc Phần mềm

Kiến trúc phần mềm là tập hợp các cấu trúc cần thiết để lập luận về hệ thống, bao gồm các thành phần phần mềm, mối quan hệ giữa chúng và các thuộc tính của cả hai. Kiến trúc thể hiện các quyết định thiết kế cấp cao nhất, khó thay đổi nhất và có ảnh hưởng lớn nhất đến khả năng đáp ứng các thuộc tính chất lượng (quality attributes) như hiệu năng, bảo mật, khả năng mở rộng và khả năng bảo trì.

2. Mô tả Kiến trúc Phần mềm

Kiến trúc được mô tả thông qua các khung nhìn (views) khác nhau - ví dụ khung nhìn logic, khung nhìn triển khai, khung nhìn tiến trình - thường theo các mô hình chuẩn hóa như ISO/IEC/IEEE 42010. Các mẫu kiến trúc (architectural patterns/styles) phổ biến bao gồm kiến trúc phân lớp (layered), microservices, hướng sự kiện (event-driven), pipe-and-filter, và kiến trúc client-server.

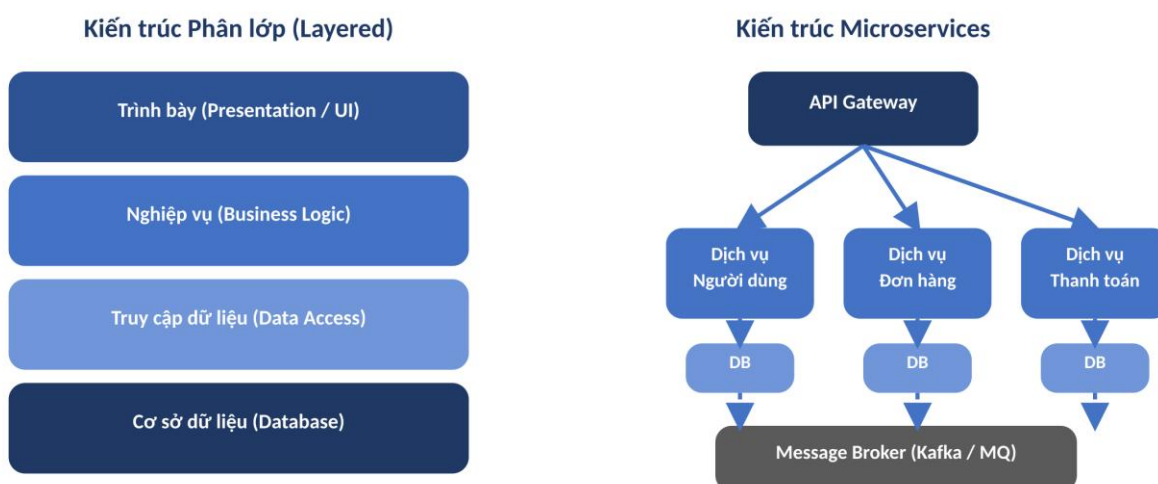
3. Quy trình Kiến trúc Phần mềm

Bao gồm các hoạt động thiết kế kiến trúc, tài liệu hóa, đánh giá và tiến hóa kiến trúc theo thời gian. Quy trình kiến trúc hiện đại thường lặp và tăng dần (iterative), gắn chặt với các chu kỳ phát triển Agile và DevOps, đòi hỏi kiến trúc sư liên tục cân bằng giữa các thuộc tính chất lượng cạnh tranh nhau (trade-off analysis).

4. Đánh giá Kiến trúc Phần mềm

Là hoạt động thẩm định xem một kiến trúc đề xuất có đáp ứng được các thuộc tính chất lượng mong muốn hay không, thông qua các phương pháp như ATAM (Architecture Tradeoff Analysis Method), rà soát kiến trúc bởi chuyên gia, và phân tích kịch bản sử dụng để kiểm tra tính khả thi trước khi đầu tư vào triển khai chi tiết.

Minh họa và Ví dụ Thực tế



Hình 2.1 - So sánh kiến trúc phân lớp truyền thống và kiến trúc microservices hiện đại.

YAML - Định nghĩa hợp đồng API giữa các microservice (OpenAPI 3.1, chuẩn 2026)

Ví dụ 2.1 - Trích đoạn đặc tả OpenAPI cho một dịch vụ trong kiến trúc microservices, dùng để sinh mã tự động (client SDK), kiểm tra hợp đồng (contract testing) và tài liệu hóa API:

```
openapi: 3.1.0
info:
  title: Order Service API
  version: 2.3.0
paths:
  /orders/{orderId}:
    get:
      summary: Lấy thông tin đơn hàng theo ID
      parameters:
        - name: orderId
          in: path
          required: true
          schema: { type: string, format: uuid }
      responses:
        '200':
          description: Thông tin đơn hàng
          content:
            application/json:
              schema:
                type: object
                properties:
                  id: { type: string, format: uuid }
                  status:
                    type: string
                    enum: [PENDING, PAID, SHIPPED, CANCELLED]
                  totalAmount: { type: number }
        '404':
          description: Không tìm thấy đơn hàng
```

Bài tập Thực hành

Bài 1. Vẽ lại (trên giấy hoặc công cụ vẽ) kiến trúc của một hệ thống bạn quen thuộc (ví dụ website thương mại điện tử cỡ nhỏ) theo phong cách microservices, xác định ít nhất 4 dịch vụ và dữ liệu chúng sở hữu riêng.

Bài 2. So sánh chi phí vận hành (operational cost) giữa kiến trúc Layered monolith và Microservices cho một đội ngũ chỉ có 3 lập trình viên. Kiến trúc nào phù hợp hơn ở giai đoạn khởi nghiệp (startup)? Vì sao?

Bài 3. Viết đặc tả OpenAPI tối giản (1 endpoint) cho một dịch vụ 'User Service' với thao tác lấy thông tin người dùng theo ID.

CHƯƠNG 3

Thiết kế Phần mềm

Software Design

Thiết kế phần mềm là quá trình chuyển hóa yêu cầu và kiến trúc thành các mô tả chi tiết đủ để lập trình viên có thể triển khai, tập trung vào cấu trúc bên trong của các thành phần phần mềm ở mức module, lớp và giao diện.

Góc nhìn chuyên sâu: các mẫu thiết kế (design patterns) là công cụ, không phải mục tiêu. Một lỗi thường gặp ở lập trình viên mới là cố nhét mẫu thiết kế vào mọi bài toán ("pattern-itis"), khiến mã nguồn trở nên phức tạp không cần thiết. Nguyên tắc "YAGNI" (You Aren't Gonna Need It) nhắc nhở rằng chỉ nên áp dụng một mẫu thiết kế khi thực sự có nhu cầu linh hoạt hoặc mở rộng trong tương lai gần, được minh chứng bằng yêu cầu cụ thể - không phải vì "có thể sẽ cần". Với AI hỗ trợ sinh mã ngày càng phổ biến năm 2026, kỹ năng quan trọng nhất của kỹ sư thiết kế không còn là ghi nhớ tên các mẫu thiết kế, mà là khả năng đánh giá xem một cấu trúc mã (dù do người hay AI sinh ra) có thực sự phù hợp với bài toán và dễ bảo trì hay không.

1. Nền tảng Thiết kế Phần mềm

Thiết kế xác định cấu trúc, thành phần, giao diện và các đặc tính khác của một hệ thống hoặc thành phần. Các nguyên lý thiết kế nền tảng bao gồm phân rã (decomposition), trừu tượng hóa (abstraction), tính module hóa, tính đóng gói (encapsulation), tính liên kết cao tính phụ thuộc thấp (high cohesion low coupling).

2. Quy trình Thiết kế Phần mềm

Bao gồm việc chuyển đổi từ mô hình phân tích/kiến trúc sang mô hình thiết kế chi tiết, ra quyết định thiết kế và ghi lại các quyết định đó. Quy trình có thể theo hướng cấu trúc (structured design), hướng đối tượng (object-oriented design) hoặc hướng thành phần.

3. Chất lượng Thiết kế Phần mềm

Các thuộc tính chất lượng thiết kế cần cân nhắc gồm: khả năng bảo trì, khả năng tái sử dụng, hiệu năng, độ tin cậy, khả năng kiểm thử và khả năng mở rộng. Các chỉ số đo lường (metrics) như độ phức tạp cyclomatic, mức độ liên kết/phụ thuộc thường được dùng để đánh giá.

4. Ghi lại Thiết kế Phần mềm

Thiết kế được ghi lại thông qua các ký hiệu (notation) chuẩn hóa như UML (biểu đồ lớp, biểu đồ trình tự, biểu đồ trạng thái), sơ đồ luồng dữ liệu, hoặc các ngôn ngữ mô tả thiết kế chuyên biệt, nhằm bảo đảm khả năng giao tiếp giữa các thành viên nhóm.

5. Chiến lược và Phương pháp Thiết kế Phần mềm

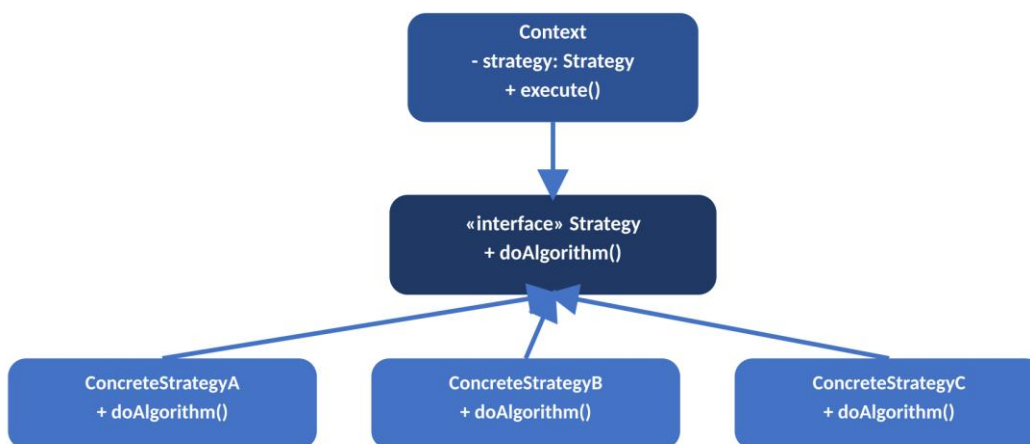
Bao gồm các phương pháp thiết kế hướng chức năng, hướng dữ liệu, hướng đối tượng, thiết kế theo thành phần (component-based design) và các mẫu thiết kế (design patterns) kinh điển như Singleton, Factory, Observer, Strategy giúp giải quyết các vấn đề thiết kế lặp lại một cách đã được kiểm chứng.

6. Phân tích và Đánh giá Chất lượng Thiết kế

Sử dụng các kỹ thuật rà soát thiết kế (design review), phân tích số liệu thiết kế (design metrics), và mô phỏng để xác minh thiết kế trước khi chuyển sang giai đoạn xây dựng, giảm thiểu rủi ro phải sửa đổi tốn kém ở giai đoạn sau.

Minh họa và Ví dụ Thực tế

Mẫu thiết kế Strategy (UML Class Diagram)



Hình 3.1 - Biểu đồ lớp UML minh họa mẫu thiết kế Strategy.

Python 3.13 - Triển khai mẫu thiết kế Strategy với type hints hiện đại

Ví dụ 3.1 - Cài đặt mẫu thiết kế Strategy bằng Python, sử dụng Protocol (typing) thay vì abstract class - phong cách được khuyến nghị trong Python hiện đại để đạt "structural typing":

```
from typing import Protocol
from dataclasses import dataclass

class PricingStrategy(Protocol):
    def calculate(self, base_price: float) -> float: ...

class RegularPricing:
    def calculate(self, base_price: float) -> float:
        return base_price

class VipDiscountPricing:
    def calculate(self, base_price: float) -> float:
        return base_price * 0.85 # giảm 15% cho khách VIP

class FlashSalePricing:
    def calculate(self, base_price: float) -> float:
        return base_price * 0.5 # giảm 50% giờ vàng

@dataclass
class ShoppingCart:
    strategy: PricingStrategy
    base_price: float

    def final_price(self) -> float:
        return self.strategy.calculate(self.base_price)

# Sử dụng - có thể đổi chiến lược lúc runtime mà không sửa ShoppingCart
cart = ShoppingCart(strategy=VipDiscountPricing(), base_price=500_000)
print(f"Giá cuối: {cart.final_price():.0f} VND")
```

TypeScript - Mẫu thiết kế Observer (thường dùng trong hệ thống sự kiện/thông báo)

Ví dụ 3.2 - Mẫu thiết kế Observer, nền tảng của các hệ thống phản ứng (reactive systems) và kiến trúc hướng sự kiện phổ biến trong ứng dụng 2026:

```
type Listener<T> = (payload: T) => void;

class EventEmitter<T> {
  private listeners: Listener<T>[] = [];

  subscribe(listener: Listener<T>): () => void {
    this.listeners.push(listener);
    return () => {
      this.listeners = this.listeners.filter((l) => l !== listener);
    };
  }

  emit(payload: T): void {
    this.listeners.forEach((listener) => listener(payload));
  }
}

interface OrderPaidEvent { orderId: string; amount: number; }

const orderPaid = new EventEmitter<OrderPaidEvent>();

// Dịch vụ Email và dịch vụ Kho hàng đều lắng nghe cùng sự kiện
orderPaid.subscribe((e) => sendReceiptEmail(e.orderId));
orderPaid.subscribe((e) => reserveInventory(e.orderId));
```

Bài tập Thực hành

Bài 1. Cài đặt mẫu thiết kế Factory Method bằng ngôn ngữ bạn quen thuộc, cho bài toán tạo ra các đối tượng 'thông báo' (Email, SMS, Push Notification) tùy theo cấu hình người dùng.

Bài 2. Đo độ phức tạp cyclomatic của một hàm bạn từng viết (dùng công cụ như radon cho Python hoặc ESLint complexity rule cho JS). Nếu > 10 , hãy đề xuất cách tái cấu trúc để giảm độ phức tạp.

Bài 3. So sánh 2 cách thiết kế: (a) một lớp Order khổng lồ tự xử lý tính giá, gửi email, cập nhật kho; (b) tách các trách nhiệm đó thành các lớp riêng. Áp dụng nguyên tắc Single Responsibility để giải thích lựa chọn tốt hơn.

CHƯƠNG 4

Xây dựng Phần mềm

Software Construction

Xây dựng phần mềm (construction) bao gồm việc tạo ra phần mềm hoạt động được thông qua lập trình chi tiết, kiểm chứng, kiểm thử đơn vị, tích hợp và gỡ lỗi - đây là hoạt động trực tiếp biến thiết kế thành mã nguồn.

Góc nhìn chuyên sâu: chất lượng xây dựng phần mềm ngày càng được đo lường không chỉ qua việc mã có chạy đúng hay không, mà qua tốc độ một lập trình viên mới có thể hiểu và sửa đổi an toàn đoạn mã đó. Nghiên cứu trong ngành liên tục chỉ ra rằng lập trình viên dành phần lớn thời gian (ước tính 60-70%) để đọc mã hiện có hơn là viết mã mới - do đó khả năng đọc hiểu (readability) quan trọng ngang hàng với tính đúng đắn. Trong môi trường 2026 với các trợ lý AI hỗ trợ viết mã (Copilot, Claude Code...), nguyên tắc này càng quan trọng hơn: mã nguồn sạch, có cấu trúc rõ ràng giúp cả con người lẫn AI hiểu ngữ cảnh chính xác hơn khi đề xuất thay đổi, giảm rủi ro AI "đoán sai" ý định của lập trình viên.

1. Nền tảng Xây dựng Phần mềm

Bao gồm các nguyên lý cơ bản như giảm thiểu độ phức tạp, dự đoán thay đổi, xây dựng để kiểm chứng được (constructing for verification) và tuân thủ các chuẩn mã hóa. Chất lượng của quá trình xây dựng ảnh hưởng trực tiếp đến số lượng lỗi và chi phí bảo trì về sau.

2. Quản lý Xây dựng

Bao gồm việc lập kế hoạch xây dựng, đo lường tiến độ, quản lý mô hình xây dựng (ví dụ xây dựng tăng dần, xây dựng theo kiểu test-driven) và phối hợp giữa các thành viên nhóm phát triển.

3. Các Cân nhắc Thực tiễn

Gồm các thực hành như lập trình phòng thủ (defensive programming), xử lý ngoại lệ, quản lý tài nguyên, tối ưu hóa mã nguồn hợp lý (không tối ưu hóa sớm), viết mã dễ đọc và tuân thủ chuẩn coding convention của tổ chức.

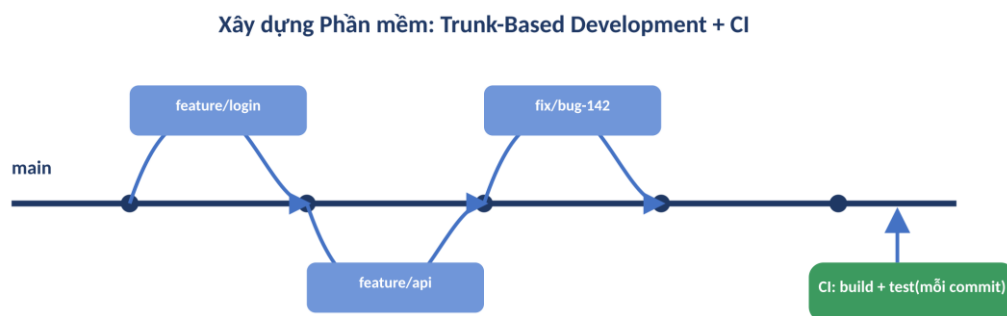
4. Công nghệ Xây dựng

Bao gồm các kỹ thuật lập trình theo ngôn ngữ (hướng đối tượng, hướng hàm, kịch bản), lập trình cặp (pair programming), tích hợp liên tục (continuous integration), quản lý cấu hình mã nguồn, và các thực hành kiểm thử đơn vị tự động gắn liền với DevOps hiện đại.

5. Công cụ Xây dựng Phần mềm

Bao gồm môi trường phát triển tích hợp (IDE), trình biên dịch/thông dịch, công cụ debug, công cụ phân tích mã tĩnh (static analysis), hệ thống quản lý phiên bản (Git) và các nền tảng CI/CD tự động hóa việc build, test và triển khai.

Minh họa và Ví dụ Thực tế



Hình 4.1 - Trunk-Based Development kết hợp tích hợp liên tục (CI) cho mỗi commit.

TypeScript - Mã sạch, dễ đọc theo chuẩn xây dựng phần mềm hiện đại

Ví dụ 4.1 - Minh họa các nguyên tắc xây dựng phần mềm tốt: đặt tên rõ ràng, tách hàm nhỏ, xử lý lỗi tường minh, tránh magic number:

```
// order-pricing.service.ts
const VIP_DISCOUNT_RATE = 0.15;
const FREE_SHIPPING_THRESHOLD = 500_000;

interface OrderLine {
  unitPrice: number;
  quantity: number;
}

export function calculateSubtotal(lines: OrderLine[]): number {
  return lines.reduce((sum, l) => sum + l.unitPrice * l.quantity, 0);
}

export function applyVipDiscount(subtotal: number, isVip: boolean): number {
  {
    if (subtotal < 0) {
      throw new RangeError('Subtotal không thể âm');
    }
    return isVip ? subtotal * (1 - VIP_DISCOUNT_RATE) : subtotal;
  }
}

export function calculateShippingFee(subtotal: number): number {
  return subtotal >= FREE_SHIPPING_THRESHOLD ? 0 : 30_000;
}
```

YAML - GitHub Actions: pipeline CI chạy cho mọi commit vào trunk

Ví dụ 4.2 - Cấu hình CI tối giản, tiêu biểu cho thực hành Trunk-Based Development năm 2026 (lint → test → build, chạy song song để rút ngắn thời gian phản hồi):

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '22' }
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
      - run: npm run build
```


TypeScript - Feature Flag: triển khai code chưa hoàn thiện an toàn vào trunk

Ví dụ 4.3 - Kỹ thuật Feature Toggle, cho phép merge code thường xuyên vào trunk (đúng tinh thần Trunk-Based Development) mà không phơi bày tính năng dở dang cho người dùng cuối:

```
interface FeatureFlags {
  newCheckoutFlow: boolean;
}

function getFeatureFlags(userId: string): FeatureFlags {
  // Trong thực tế: gọi dịch vụ flag như LaunchDarkly / Unleash
  return { newCheckoutFlow: isInRolloutGroup(userId, 'new-checkout', 0.1) };
}

function renderCheckout(userId: string) {
  const flags = getFeatureFlags(userId);
  return flags.newCheckoutFlow
    ? renderNewCheckoutFlow() // đang phát triển, rollout 10%
    : renderLegacyCheckoutFlow();
}
```

Bài tập Thực hành

Bài 1. Thiết lập một pipeline CI thật (dùng GitHub Actions hoặc GitLab CI) cho một repo cá nhân nhỏ: chạy lint + test tự động mỗi khi push. Chụp lại ảnh màn hình pipeline chạy thành công.

Bài 2. Refactor một hàm dài (>40 dòng) trong một project cũ của bạn thành các hàm nhỏ hơn, mỗi hàm chỉ làm một việc. So sánh độ dễ đọc trước/sau.

Bài 3. Thiết kế một feature flag đơn giản (dùng biến môi trường hoặc file cấu hình) để bật/tắt một tính năng mới trong ứng dụng của bạn mà không cần deploy lại.

CHƯƠNG 5

Nền tảng Kiểm thử Phần mềm

Software Testing

Kiểm thử phần mềm là quá trình đánh giá động (dynamic) một hệ thống hoặc thành phần bằng cách thực thi nó, nhằm phát hiện khiếm khuyết và cung cấp bằng chứng về chất lượng, độ tin cậy của sản phẩm.

Góc nhìn chuyên sâu: kiểm thử không phải là hoạt động chỉ diễn ra sau khi viết mã - triết lý "shift-left testing" (dịch chuyển kiểm thử về sớm) khẳng định rằng viết test trước hoặc song song với code (như trong Test-Driven Development) giúp phát hiện lỗi thiết kế ngay từ đầu, khi chi phí sửa còn thấp (xem thêm đường cong chi phí ở Hình 15.1). Một cạm bẫy phổ biến là chạy theo chỉ số độ bao phủ mã nguồn (code coverage) như mục tiêu tối thượng - 100% coverage không đảm bảo không có lỗi nếu các assertion trong test quá yếu hoặc không kiểm tra đúng logic nghiệp vụ. Coverage chỉ nên được xem là chỉ báo cảnh giác (phần code nào chưa từng được thực thi bởi test), không phải thước đo chất lượng tuyệt đối.

1. Nền tảng Kiểm thử Phần mềm

Kiểm thử nhằm phát hiện lỗi (defect) chứ không chứng minh sự vắng mặt hoàn toàn của lỗi. Các khái niệm nền tảng gồm: lỗi (fault), sai sót (error), hư hỏng (failure), độ bao phủ kiểm thử (test coverage), và nguyên tắc kiểm thử càng sớm càng tốt (shift-left testing).

2. Mức Kiểm thử (Test Levels)

Bao gồm kiểm thử đơn vị (unit testing) kiểm tra từng thành phần riêng lẻ, kiểm thử tích hợp (integration testing) kiểm tra tương tác giữa các thành phần, kiểm thử hệ thống (system testing) kiểm tra toàn bộ hệ thống theo yêu cầu, và kiểm thử chấp nhận (acceptance testing) xác nhận hệ thống đáp ứng nhu cầu người dùng cuối.

3. Kỹ thuật Kiểm thử (Test Techniques)

Gồm kiểm thử hộp đen (black-box, dựa trên đặc tả), kiểm thử hộp trắng (white-box, dựa trên cấu trúc mã nguồn), kiểm thử dựa trên kinh nghiệm (experience-based) và các kỹ thuật kết hợp như phân vùng tương đương, phân tích giá trị biên, kiểm thử dựa trên mô hình (model-based testing).

4. Các Độ đo liên quan đến Kiểm thử

Bao gồm độ bao phủ mã nguồn (code coverage), mật độ lỗi (defect density), hiệu quả loại bỏ lỗi (defect removal efficiency) và các chỉ số dùng để đánh giá mức độ đầy đủ và hiệu quả của quá trình kiểm thử.

5. Quy trình Kiểm thử

Bao gồm lập kế hoạch kiểm thử, thiết kế test case, thực thi kiểm thử, ghi nhận và theo dõi lỗi, và đánh giá tiêu chí hoàn thành kiểm thử (exit criteria).

6. Kiểm thử trong Các Quy trình Phát triển và Miền Ứng dụng

Cách tiếp cận kiểm thử thay đổi tùy theo mô hình phát triển (thác nước, Agile, DevOps) và miền ứng dụng (hệ thống nhúng, hệ thống an toàn nghiêm ngặt, ứng dụng web, di động), mỗi bối cảnh có yêu cầu và ràng buộc riêng.

7. Kiểm thử của và thông qua Công nghệ Mới nổi

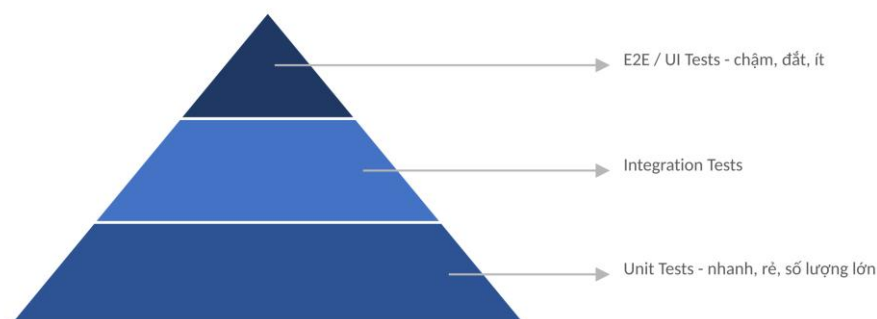
Đề cập đến các thách thức kiểm thử đặc thù của hệ thống trí tuệ nhân tạo/học máy (kiểm thử tính công bằng, độ bền vững của mô hình), hệ thống dựa trên đám mây, và việc sử dụng chính các công nghệ mới nổi (như AI) để hỗ trợ tự động hóa kiểm thử.

8. Công cụ Kiểm thử Phần mềm

Bao gồm framework kiểm thử tự động (JUnit, Selenium, Cypress), công cụ quản lý test case, công cụ phân tích độ bao phủ và các nền tảng kiểm thử hiệu năng, kiểm thử bảo mật chuyên dụng.

Minh họa và Ví dụ Thực tế

Kim tự tháp Kiểm thử (Test Pyramid)



Hình 5.1 - Kim tự tháp kiểm thử: nhiều unit test, ít end-to-end test.

Python - pytest: unit test + kiểm thử dựa trên thuộc tính (property-based testing)

Ví dụ 5.1 - Kết hợp unit test truyền thống với kiểm thử dựa trên thuộc tính (property-based testing, dùng thư viện Hypothesis) - kỹ thuật ngày càng phổ biến năm 2026 để phát hiện các trường hợp biên mà con người khó nghĩ tới:

```
import pytest
from hypothesis import given, strategies as st
from pricing import apply_vip_discount

def test_vip_discount_applies_15_percent():
    assert apply_vip_discount(100_000, is_vip=True) == 85_000

def test_regular_customer_no_discount():
    assert apply_vip_discount(100_000, is_vip=False) == 100_000

def test_negative_subtotal_raises_error():
    with pytest.raises(ValueError):
        apply_vip_discount(-1, is_vip=True)

# Property-based test: đúng với MỌI subtotal hợp lệ, không chỉ vài ví dụ
@given(subtotal=st.floats(min_value=0, max_value=1_000_000_000))
def test_discounted_price_never_exceeds_original(subtotal):
    discounted = apply_vip_discount(subtotal, is_vip=True)
    assert 0 <= discounted <= subtotal
```

TypeScript - Playwright: kiểm thử end-to-end (E2E) cho luồng thanh toán

Ví dụ 5.2 - Kiểm thử E2E mô phỏng hành vi người dùng thật trên trình duyệt, nằm ở đỉnh kim tự tháp (ít nhưng có giá trị cao vì kiểm tra toàn bộ luồng tích hợp):

```
import { test, expect } from '@playwright/test';

test('khách hàng có thể hoàn tất thanh toán bằng ví điện tử', async ({ page }) => {
    await page.goto('/cart');
    await page.click('text=Tiến hành thanh toán');
    await page.selectOption('#payment-method', 'e-wallet');
    await page.click('text=Xác nhận thanh toán');

    await expect(page.locator('.order-status')).toHaveText('Đã thanh toán');
    await expect(page.locator('.toast-success')).toBeVisible();
});
```

Bài tập Thực hành

Bài 1. Viết 5 unit test cho một hàm tính điểm giảm giá bậc thang (chú ý các trường hợp biên: 0, âm, vừa đủ ngưỡng, vượt ngưỡng, số lớn).

Bài 2. Cài đặt Hypothesis (Python) hoặc fast-check (TypeScript) cho một hàm bạn tự viết, thử tìm ít nhất 1 trường hợp biên bạn chưa từng nghĩ tới.

Bài 3. Tính tỷ lệ hiện tại giữa số lượng unit test / integration test / E2E test trong một dự án bạn có (hoặc dự án mã nguồn mở bạn biết). So sánh với hình dạng lý tưởng của kim tự tháp kiểm thử và đề xuất điều chỉnh.

CHƯƠNG 6

Vận hành KỸ THUẬT PHẦN MỀM

Software Engineering Operations

Đây là lĩnh vực tri thức mới thứ hai được bổ sung trong SWEBOK V4.0, phản ánh xu hướng DevOps và SRE (Site Reliability Engineering) hiện đại, trong đó việc vận hành phần mềm trong môi trường sản xuất trở thành một phần không thể tách rời của KỸ THUẬT PHẦN MỀM chứ không còn là hoạt động riêng biệt sau khi bàn giao.

Góc nhìn chuyên sâu: văn hóa DevOps thực chất là một thay đổi về tổ chức và con người nhiều hơn là công cụ. Nhiều tổ chức mua sắm công cụ CI/CD hiện đại nhưng vẫn giữ nguyên silo tổ chức (đội Dev và đội Ops tách biệt, đổ lỗi cho nhau khi có sự cố) - kết quả là tốc độ triển khai không cải thiện đáng kể dù có công cụ tốt. DevOps thành công đòi hỏi trách nhiệm chung ("you build it, you run it"): đội phát triển tham gia trực tiếp vào vận hành và ứng phó sự cố sản phẩm của chính họ, tạo động lực viết mã dễ vận hành và giám sát tốt ngay từ đầu. Văn hóa "blameless postmortem" (phân tích sự cố không đổ lỗi cá nhân) là yếu tố then chốt để đội ngũ dám báo cáo sự cố sớm và học hỏi thay vì che giấu.

1. Nền tảng Vận hành KỸ THUẬT PHẦN MỀM

Vận hành phần mềm bao gồm các hoạt động bảo đảm hệ thống phần mềm chạy ổn định, tin cậy trong môi trường thực tế, gắn kết chặt chẽ giữa đội phát triển (Dev) và đội vận hành (Ops) theo triết lý DevOps, với mục tiêu rút ngắn chu kỳ phát hành và tăng độ tin cậy dịch vụ.

2. Lập Kế hoạch Vận hành

Bao gồm việc xác định chiến lược triển khai, lập kế hoạch năng lực (capacity planning), thiết kế cho khả năng vận hành (operability), và chuẩn bị các quy trình ứng phó sự cố trước khi hệ thống đi vào sản xuất.

3. Cung cấp Dịch vụ Vận hành

Bao gồm các hoạt động triển khai liên tục (continuous deployment), quản lý phát hành, tự động hóa hạ tầng (infrastructure as code), và giám sát trạng thái hệ thống trong thời gian thực.

4. Kiểm soát Vận hành

Bao gồm giám sát (monitoring), ghi nhật ký (logging), cảnh báo (alerting), quản lý sự cố (incident management) và phân tích nguyên nhân gốc rễ (root cause analysis) khi có sự cố xảy ra, nhằm duy trì các chỉ số dịch vụ như SLA/SLO.

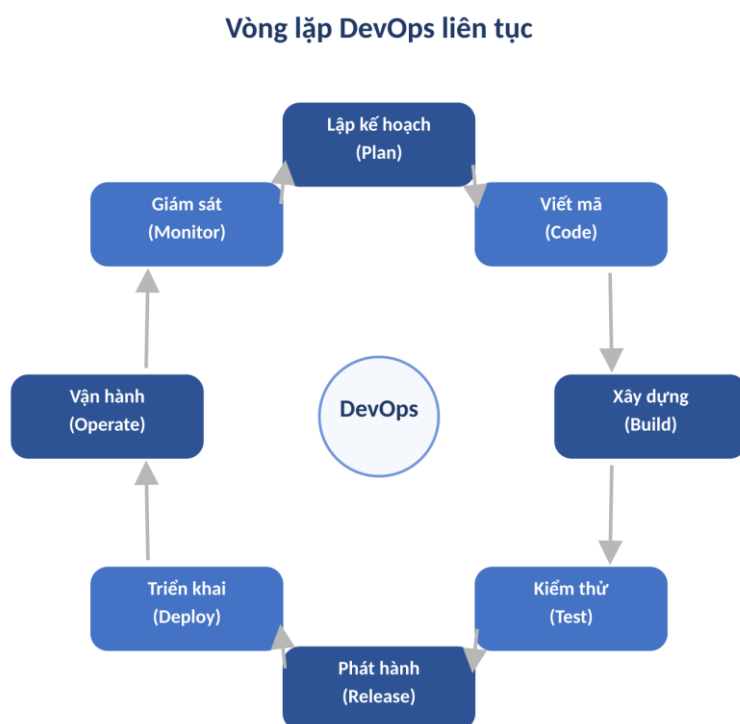
5. Các Cân nhắc Thực tiễn

Bao gồm quản lý rủi ro vận hành, cân bằng giữa tốc độ phát hành và độ ổn định, văn hóa "blameless postmortem" khi phân tích sự cố, và quản lý nợ kỹ thuật (technical debt) trong vận hành lâu dài.

6. Công cụ Vận hành KỸ THUẬT PHẦN MỀM

Bao gồm các nền tảng container hóa và điều phối (Docker, Kubernetes), công cụ giám sát (Prometheus, Grafana), công cụ quản lý cấu hình hạ tầng (Terraform, Ansible) và các nền tảng CI/CD.

Minh họa và Ví dụ Thực tế



Hình 6.1 - Vòng lặp DevOps liên tục: Plan–Code–Build–Test–Release–Deploy–Operate–Monitor.

Dockerfile - Đóng gói dịch vụ theo chuẩn container tối giản, đa giai đoạn (multi-stage)

Ví dụ 6.1 - Dockerfile đa giai đoạn (multi-stage build) giúp giảm kích thước image production, thực hành chuẩn cho vận hành KỸ THUẬT PHẦN MỀM hiện đại:

```
# --- Giai đoạn build ---
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build

# --- Giai đoạn chạy (production, tối giản) ---
FROM gcr.io/distroless/nodejs22-debian12 AS runtime
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER nonroot
EXPOSE 8080
CMD ["dist/server.js"]
```


YAML - Kubernetes Deployment với health check và giới hạn tài nguyên

Ví dụ 6.2 - Manifest triển khai lên Kubernetes, bao gồm readiness/liveness probe và resource limits - bắt buộc trong vận hành production năm 2026:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3
  selector:
    matchLabels: { app: order-service }
  template:
    metadata:
      labels: { app: order-service }
    spec:
      containers:
        - name: order-service
          image: registry.internal/order-service:2.3.0
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: '200m', memory: '256Mi' }
            limits: { cpu: '500m', memory: '512Mi' }
          readinessProbe:
            httpGet: { path: /healthz/ready, port: 8080 }
            initialDelaySeconds: 5
          livenessProbe:
            httpGet: { path: /healthz/live, port: 8080 }
            periodSeconds: 10
```

PromQL - Truy vấn cảnh báo tỷ lệ lỗi cao (Prometheus alerting rule)

Ví dụ 6.3 - Luật cảnh báo giám sát: tự động thông báo khi tỷ lệ lỗi HTTP 5xx vượt ngưỡng, thực hành cốt lõi của kiểm soát vận hành (Operations Control):

```
groups:
- name: order-service-alerts
  rules:
  - alert: HighErrorRate
    expr: >
      sum(rate(http_requests_total{status=~"5.."}[5m]))
      / sum(rate(http_requests_total[5m])) > 0.02
    for: 5m
    labels: { severity: critical }
    annotations:
      summary: "Tỷ lệ lỗi 5xx > 2% trong 5 phút liên tục"
      runbook: "https://wiki.internal/runbooks/high-error-rate"
```

Bài tập Thực hành

Bài 1. Viết một Dockerfile đơn giản (single-stage là được) để đóng gói một ứng dụng nhỏ bạn từng viết. Build và chạy thử container đó cục bộ.

Bài 2. Thiết kế 2 endpoint /healthz/ready và /healthz/live cho một API bất kỳ, giải thích sự khác biệt về mục đích của hai endpoint này trong vận hành Kubernetes.

Bài 3. Liệt kê 3 chỉ số (metric) quan trọng nhất bạn sẽ giám sát cho một dịch vụ thanh toán trực tuyến, và mức ngưỡng (threshold) hợp lý để cảnh báo cho mỗi chỉ số.

CHƯƠNG 7

Bảo trì Phần mềm

Software Maintenance

Bảo trì phần mềm là toàn bộ các hoạt động thay đổi phần mềm sau khi bàn giao, nhằm sửa lỗi, cải thiện hiệu năng hoặc các thuộc tính khác, hay thích nghi phần mềm với môi trường đã thay đổi.

Góc nhìn chuyên sâu: bảo trì thường bị đánh giá thấp về mặt danh tiếng nghề nghiệp so với phát triển tính năng mới, dù chiếm phần lớn ngân sách vòng đời phần mềm trong thực tế. Hệ quả là nhiều tổ chức phân công kỹ sư ít kinh nghiệm nhất vào công việc bảo trì - một quyết định phản tác dụng, vì hiểu và sửa an toàn một hệ thống phức tạp đã tồn tại thường đòi hỏi kinh nghiệm sâu hơn cả việc viết mã mới từ đầu. Các tổ chức trưởng thành đầu tư nghiêm túc vào bảo trì phòng ngừa (preventive maintenance - ví dụ nâng cấp dependency định kỳ, tái cấu trúc chủ động) để giảm tích lũy nợ kỹ thuật (technical debt), thay vì chỉ phản ứng khi sự cố xảy ra.

1. Nền tảng Bảo trì Phần mềm

Các loại bảo trì kinh điển gồm: bảo trì sửa lỗi (corrective), bảo trì thích nghi (adaptive thích ứng với thay đổi môi trường), bảo trì hoàn thiện (perfective cải thiện hiệu năng/khả năng bảo trì) và bảo trì phòng ngừa (preventive).

2. Các Vấn đề Then chốt trong Bảo trì

Bao gồm thách thức về chi phí bảo trì thường chiếm phần lớn tổng chi phí vòng đời phần mềm, vấn đề hiểu mã nguồn cũ (legacy code), quản lý tri thức khi nhân sự thay đổi, và cân bằng giữa việc sửa nhanh và giữ chất lượng lâu dài.

3. Quy trình Bảo trì Phần mềm

Bao gồm tiếp nhận yêu cầu thay đổi, phân tích tác động (impact analysis), lập kế hoạch phát hành bảo trì, thực hiện thay đổi, kiểm thử hồi quy (regression testing) và bàn giao bản cập nhật.

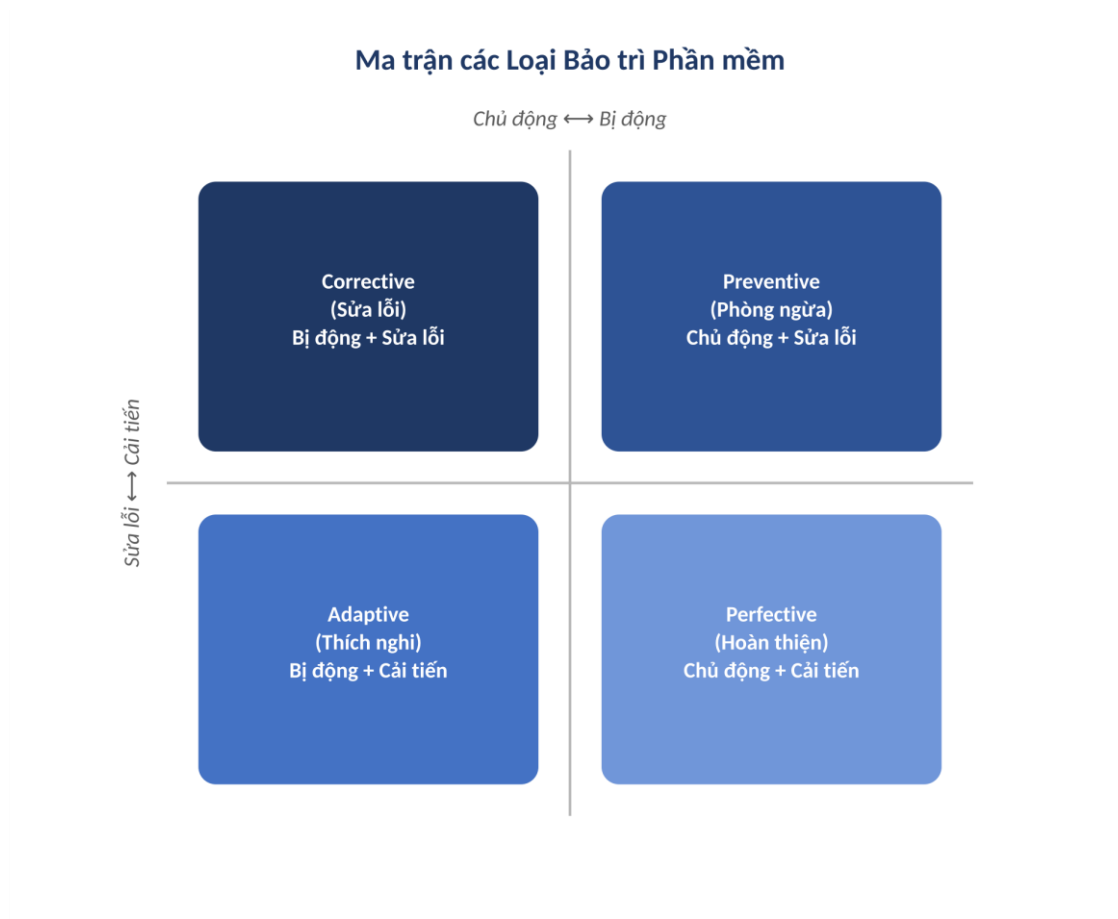
4. Kỹ thuật Bảo trì Phần mềm

Bao gồm phân tích chương trình (program comprehension), tái cấu trúc mã nguồn (refactoring), tái kỹ nghệ (reengineering), và các kỹ thuật đo lường mức độ phức tạp để ưu tiên hóa công việc bảo trì.

5. Công cụ Bảo trì Phần mềm

Bao gồm công cụ phân tích mã nguồn tĩnh/động, công cụ trực quan hóa cấu trúc hệ thống, công cụ quản lý yêu cầu thay đổi và hệ thống theo dõi lỗi (bug tracker).

Minh họa và Ví dụ Thực tế



Hình 7.1 - Ma trận bốn loại bảo trì phần mềm theo tính chủ động và mục tiêu.

Python - Refactoring: "trước" và "sau" khi bảo trì hoàn thiện (perfective maintenance)

Ví dụ 7.1 - Minh họa bảo trì hoàn thiện (perfective maintenance): cải thiện khả năng đọc và bảo trì của mã nguồn cũ mà không thay đổi hành vi bên ngoài - luôn đi kèm kiểm thử hồi quy để đảm bảo an toàn:

```
# --- TRƯỚC: mã khó đọc, khó mở rộng (legacy code điển hình) ---
def calc(o, t):
    r = 0
    for i in o:
        r += i['p'] * i['q']
    if t == 1:
        r = r * 0.85
    elif t == 2:
        r = r * 0.5
    return r

# --- SAU: đặt tên rõ ràng, tách trách nhiệm (perfective maintenance) ---
from enum import IntEnum

class CustomerTier(IntEnum):
    REGULAR = 0
    VIP = 1
    FLASH_SALE = 2

DISCOUNT_RATES = {
    CustomerTier.REGULAR: 1.0,
    CustomerTier.VIP: 0.85,
    CustomerTier.FLASH_SALE: 0.5,
}

def calculate_total(order_lines: list[dict], tier: CustomerTier) -> float:
    subtotal = sum(line['price'] * line['qty'] for line in order_lines)
    return subtotal * DISCOUNT_RATES[tier]
```

Bài tập Thực hành

Bài 1. Tìm một đoạn mã cũ (của chính bạn hoặc mã nguồn mở) khó hiểu. Phân loại xem việc sửa nó thuộc loại bảo trì nào (corrective/adaptive/perfective/preventive) và giải thích vì sao.

Bài 2. Ước lượng: nếu một hệ thống có 100.000 dòng code, tồn tại 10 năm, hãy ước lượng tỷ lệ % thời gian đội ngũ kỹ sư dành cho bảo trì so với phát triển tính năng mới.

Bài 3. Thiết kế một quy trình đơn giản (3-4 bước) để tiếp nhận và ưu tiên hóa các yêu cầu bảo trì (change request) cho một đội phát triển 5 người.

CHƯƠNG 8

Quản lý Cấu hình Phần mềm

Software Configuration Management

Quản lý cấu hình phần mềm (SCM) là hoạt động kiểm soát sự tiến hóa của các sản phẩm công việc phần mềm trong suốt vòng đời, bảo đảm tính toàn vẹn và khả năng truy vết của mọi thay đổi.

Góc nhìn chuyên sâu: quản lý cấu hình hiện đại đã mở rộng vượt xa phạm vi mã nguồn - khái niệm "Configuration as Code" giờ bao trùm cả hạ tầng (Infrastructure as Code, xem Ví dụ 8.2), cấu hình bảo mật (Policy as Code), và thậm chí cả quy trình vận hành (Runbook as Code). Nguyên tắc chung xuyên suốt là: bất kỳ trạng thái nào của hệ thống có thể ảnh hưởng đến hành vi sản xuất đều nên được lưu trong hệ thống kiểm soát phiên bản, có thể xem lại lịch sử, và có thể khôi phục (rollback) một cách đáng tin cậy. Đây là nền tảng cho khả năng phục hồi sau sự cố (disaster recovery) và tuân thủ quy định (compliance audit) trong các ngành có yêu cầu nghiêm ngặt như tài chính, y tế.

1. Quản lý Quy trình SCM

Bao gồm việc lập kế hoạch SCM, xác định vai trò trách nhiệm, và thiết lập các chính sách kiểm soát thay đổi phù hợp với quy mô và tính chất dự án.

2. Nhận dạng Cấu hình Phần mềm

Là việc xác định các hạng mục cấu hình (configuration items) cần được kiểm soát - như mã nguồn, tài liệu thiết kế, kịch bản build - và thiết lập cơ chế đặt tên, đánh phiên bản cho chúng.

3. Kiểm soát Thay đổi Cấu hình

Bao gồm quy trình đề xuất, đánh giá, phê duyệt và triển khai thay đổi đối với các hạng mục cấu hình đã được thiết lập baseline, thường thông qua hội đồng kiểm soát thay đổi (Change Control Board).

4. Kế toán Trạng thái Cấu hình

Là việc ghi nhận và báo cáo trạng thái hiện tại của các hạng mục cấu hình và lịch sử thay đổi của chúng, phục vụ mục đích kiểm toán và truy vết.

5. Kiểm toán Cấu hình Phần mềm

Bao gồm việc xác minh rằng các hạng mục cấu hình tuân thủ đúng yêu cầu đã đặc tả (kiểm toán chức năng) và phù hợp với tài liệu mô tả (kiểm toán vật lý).

6. Quản lý Phát hành và Bàn giao Phần mềm

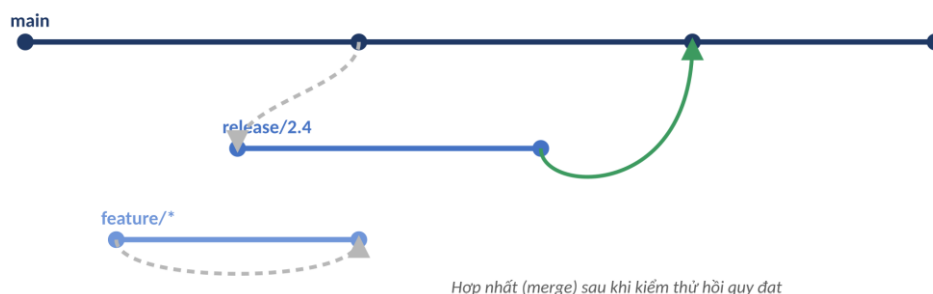
Bao gồm việc đóng gói, đánh phiên bản, lập tài liệu phát hành (release notes) và quản lý quy trình bàn giao phần mềm đến môi trường sản xuất hoặc khách hàng.

7. Công cụ Quản lý Cấu hình Phần mềm

Bao gồm hệ thống quản lý phiên bản (Git, SVN), công cụ quản lý build tự động, và các nền tảng quản lý phát hành tích hợp trong pipeline CI/CD.

Minh họa và Ví dụ Thực tế

Quản lý Cấu hình: Chiến lược nhánh Git (Release branching)



Hình 8.1 - Chiến lược nhánh Git dùng cho quản lý cấu hình theo release.

Bash - Quy ước Conventional Commits + semantic versioning tự động

Ví dụ 8.1 - Quy ước đặt tên commit chuẩn hóa (Conventional Commits), cho phép công cụ tự động sinh CHANGELOG và số phiên bản semver - thực hành phổ biến trong quản lý cấu hình hiện đại:

```
# Định dạng: <type>(<phạm vi>): <mô tả ngắn>
git commit -m "feat(payment): thêm hỗ trợ thanh toán qua ví điện tử"
git commit -m "fix(order): sửa lỗi tính sai phí vận chuyển cho đơn > 500k"
git commit -m "docs(api): cập nhật đặc tả OpenAPI cho endpoint /orders"
git commit -m "refactor(pricing): tách logic giảm giá VIP thành hàm riêng"

# feat -> tăng MINOR version (1.2.0 -> 1.3.0)
# fix -> tăng PATCH version (1.2.0 -> 1.2.1)
# BREAKING CHANGE trong footer -> tăng MAJOR version (1.2.0 -> 2.0.0)

# Công cụ semantic-release đọc lịch sử commit và tự động:
# 1) tính số phiên bản mới, 2) sinh CHANGELOG.md,
# 3) gắn tag Git, 4) publish package, 5) tạo GitHub Release
npx semantic-release
```

HCL (Terraform) - Quản lý cấu hình hạ tầng dưới dạng mã (Infrastructure as Code)

Ví dụ 8.2 - Mở rộng khái niệm quản lý cấu hình sang hạ tầng: định nghĩa tài nguyên đám mây bằng mã, được kiểm soát phiên bản giống hệt mã nguồn ứng dụng:

```
resource "aws_s3_bucket" "order_documents" {
  bucket = "company-order-documents-prod"

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Owner       = "platform-team"
  }
}

resource "aws_s3_bucket_versioning" "order_documents" {
  bucket = aws_s3_bucket.order_documents.id
  versioning_configuration { status = "Enabled" }
}

# terraform plan -> xem trước thay đổi (giống 'git diff' cho hạ tầng)
# terraform apply -> áp dụng, có audit trail đầy đủ trong Git history
```

Bài tập Thực hành

Bài 1. Áp dụng Conventional Commits cho 5 commit tiếp theo trong một repo cá nhân của bạn. Sau đó thử chạy công cụ 'standard-version' hoặc 'semantic-release' để xem CHANGELOG được sinh tự động.

Bài 2. Thiết kế một chiến lược nhánh Git phù hợp cho một đội 4 người phát hành sản phẩm mỗi 2 tuần - vẽ sơ đồ tương tự Hình 8.1 cho quy trình bạn đề xuất.

Bài 3. Giải thích sự khác biệt giữa 'kiểm toán chức năng' (functional audit) và 'kiểm toán vật lý' (physical audit) trong SCM bằng một ví dụ cụ thể bạn tự nghĩ ra.

CHƯƠNG 9

Quản lý KỸ THUẬT PHẦN MỀM

Software Engineering Management

Quản lý KỸ THUẬT PHẦN MỀM là việc áp dụng các hoạt động quản lý - lập kế hoạch, phối hợp, đo lường, giám sát, kiểm soát và báo cáo - để bảo đảm dự án phần mềm được thực hiện một cách có hệ thống, đúng tiến độ và đạt chất lượng mong muốn.

Góc nhìn chuyên sâu: các chỉ số quản lý dự án như vận tốc (velocity) rất dễ bị lạm dụng nếu dùng sai mục đích - velocity chỉ nên là công cụ dự báo nội bộ của một nhóm cụ thể, không nên dùng để so sánh giữa các nhóm khác nhau (vì mỗi nhóm ước lượng story point theo thang riêng), và tuyệt đối không nên dùng làm chỉ số đánh giá hiệu suất cá nhân - điều này khuyến khích nhóm "thối phòng" điểm số thay vì cải thiện thực chất (hiệu ứng Goodhart's Law: khi một chỉ số trở thành mục tiêu, nó không còn là thước đo tốt nữa). Quản lý dự án phần mềm hiệu quả năm 2026 tập trung nhiều hơn vào các chỉ số luồng công việc (flow metrics) như thời gian chu trình (cycle time) và tần suất triển khai (deployment frequency) - được đề xuất bởi nghiên cứu DORA - vì chúng khó bị thao túng hơn và phản ánh trực tiếp giá trị mang lại cho khách hàng.

1. Khởi động và Xác định Phạm vi

Bao gồm việc xác định mục tiêu dự án, đánh giá tính khả thi, xác định phạm vi công việc và thiết lập các tiêu chí thành công ngay từ đầu dự án.

2. Lập Kế hoạch Dự án Phần mềm

Bao gồm ước lượng công sức, lập lịch trình, phân bổ nguồn lực, lập kế hoạch quản lý rủi ro và xây dựng kế hoạch quản lý chất lượng cho dự án.

3. Thực thi Dự án Phần mềm

Bao gồm việc điều phối nhóm phát triển, quản lý các bên liên quan, thực hiện các hoạt động đã lập kế hoạch và xử lý các vấn đề phát sinh trong quá trình triển khai.

4. Rà soát và Đánh giá

Bao gồm việc theo dõi tiến độ so với kế hoạch, tổ chức các buổi rà soát định kỳ (sprint review, milestone review) và điều chỉnh kế hoạch khi cần thiết.

5. Kết thúc Dự án

Bao gồm việc bàn giao sản phẩm cuối cùng, tổng kết bài học kinh nghiệm (lessons learned/retrospective) và giải phóng nguồn lực dự án.

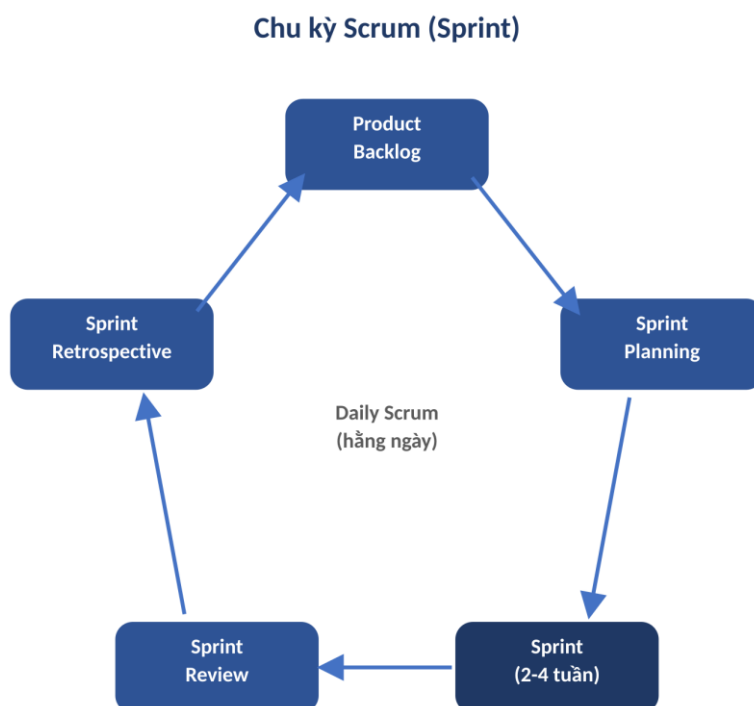
6. Đo lường KỸ THUẬT PHẦN MỀM

Bao gồm việc thiết lập chương trình đo lường, thu thập và phân tích các chỉ số dự án (năng suất, chất lượng, chi phí) để hỗ trợ ra quyết định dựa trên dữ liệu.

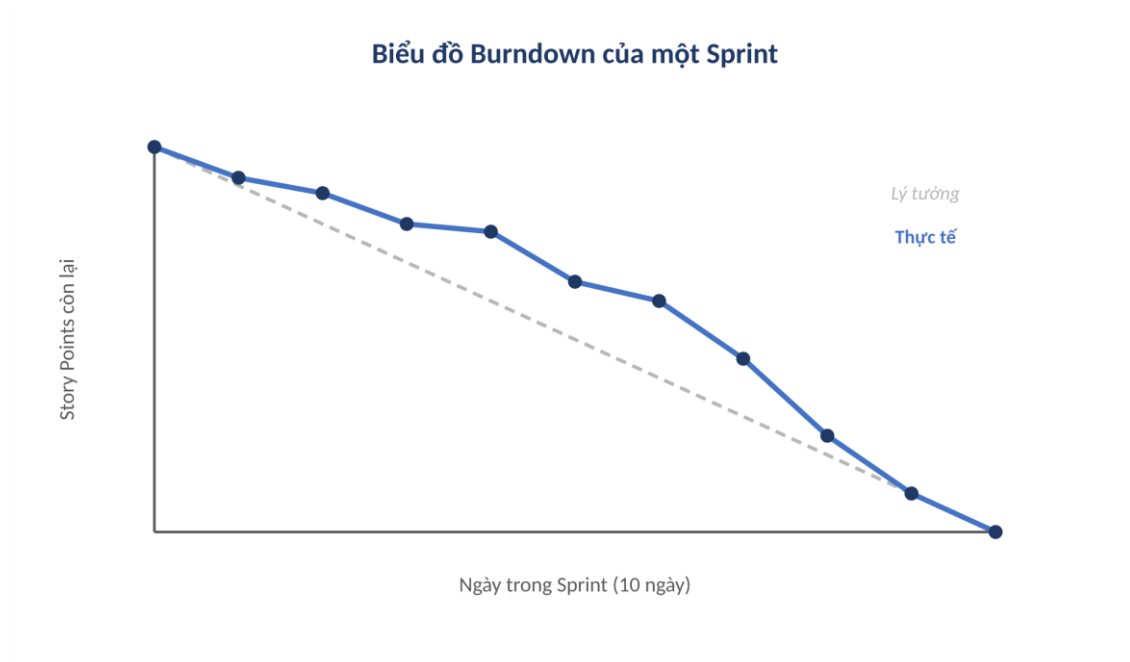
7. Công cụ Quản lý KỸ THUẬT PHẦN MỀM

Bao gồm phần mềm quản lý dự án (Jira, MS Project), bảng Kanban/Scrum điện tử, công cụ theo dõi thời gian và công cụ báo cáo trực quan hóa dữ liệu dự án.

Minh họa và Ví dụ Thực tế



Hình 9.1 - Chu kỳ Scrum: Backlog → Planning → Sprint → Review → Retrospective.



Hình 9.2 - Biểu đồ Burndown giúp trực quan hóa tiến độ hoàn thành công việc trong Sprint.

Python - Tính vận tốc nhóm (team velocity) và dự báo ngày hoàn thành backlog

Ví dụ 9.1 - Một kỹ thuật đo lường quản lý dự án phổ biến: dùng vận tốc trung bình của các sprint gần nhất để dự báo (forecast) thời điểm hoàn thành toàn bộ backlog:

```
from statistics import mean

sprint_velocities = [32, 28, 35, 30, 31] # story points hoàn thành mỗi sprint gần đây
remaining_backlog_points = 210

avg_velocity = mean(sprint_velocities)
sprints_needed = remaining_backlog_points / avg_velocity

print(f"Vận tốc trung bình: {avg_velocity:.1f} điểm/sprint")
print(f"Số sprint dự kiến còn lại: {sprints_needed:.1f}")
print(f"=> Hoàn thành sau khoảng {round(sprints_needed) * 2} tuần (sprint 2 tuần)")
```

Bài tập Thực hành

Bài 1. Tính vận tốc trung bình của 3 sprint gần nhất trong một dự án bạn tham gia (thật hoặc giả định), và dự báo thời gian hoàn thành 100 story points còn lại.

Bài 2. Vẽ biểu đồ burndown cho một sprint 5 ngày với dữ liệu tự chọn (số điểm còn lại mỗi ngày), rồi phân tích: nhóm có đang đi đúng tiến độ so với đường lý tưởng không?

Bài 3. Viết một bản 'định nghĩa hoàn thành' (Definition of Done) gồm ít nhất 5 tiêu chí cho một đội phát triển web.

CHƯƠNG 10

Quy trình KỸ THUẬT PHẦN MỀM

Software Engineering Process

Lĩnh vực này tập trung vào bản thân quy trình phần mềm như một đối tượng được định nghĩa, đo lường, kiểm soát và cải tiến một cách có hệ thống.

Góc nhìn chuyên sâu: các mô hình trưởng thành quy trình như CMMI thường bị hiểu lầm là mục tiêu "đạt chứng chỉ" thay vì công cụ cải tiến thực chất - một tổ chức có thể đạt CMMI mức 3 trên giấy tờ nhưng vẫn giao sản phẩm chậm và chất lượng kém nếu quy trình chỉ được tuân thủ hình thức. Giá trị thực sự của các mô hình trưởng thành nằm ở việc tạo ra một ngôn ngữ chung để tổ chức tự đánh giá điểm yếu và có lộ trình cải tiến rõ ràng, không phải ở tấm chứng chỉ. Trong các đội Agile/DevOps hiện đại, tinh thần trưởng thành quy trình thể hiện qua việc đo lường liên tục (không chỉ định lượng qua số liệu mà cả định tính qua retrospective) và sẵn sàng thay đổi quy trình khi có bằng chứng nó không còn hiệu quả.

1. Nền tảng Quy trình KỸ THUẬT PHẦN MỀM

Bao gồm các khái niệm về mô hình quy trình (process model), thực thi quy trình (process enactment), và các thành phần cơ bản của một quy trình phần mềm như hoạt động, vai trò, sản phẩm công việc đầu vào/đầu ra.

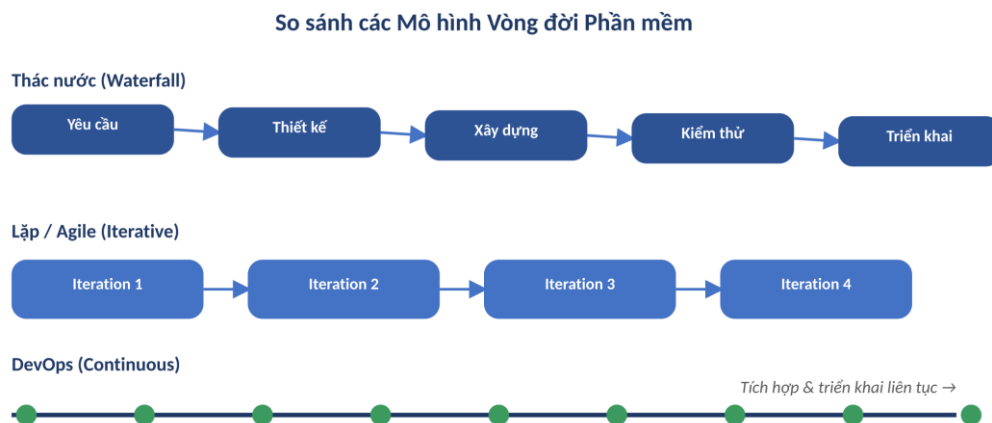
2. Vòng đời (Life Cycles)

Bao gồm các mô hình vòng đời phần mềm kinh điển như mô hình thác nước (waterfall), mô hình xoắn ốc (spiral), mô hình tăng dần/lặp (incremental/iterative) và các phương pháp Agile (Scrum, Kanban, XP) hiện đang phổ biến rộng rãi.

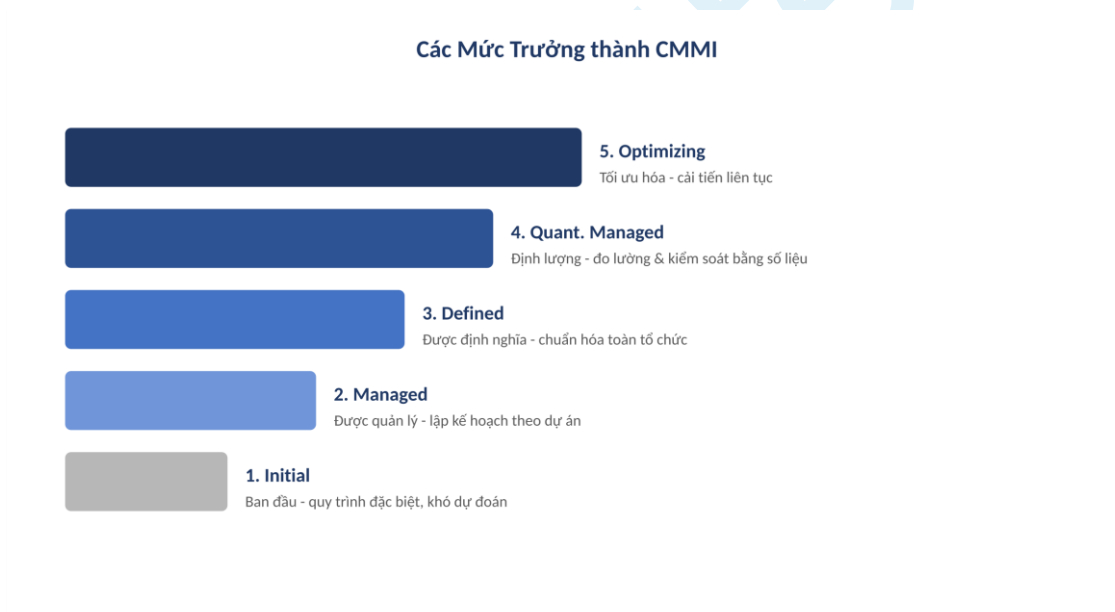
3. Đánh giá và Cải tiến Quy trình Phần mềm

Bao gồm các mô hình trưởng thành quy trình như CMMI (Capability Maturity Model Integration), ISO/IEC 15504 (SPICE), và các phương pháp cải tiến quy trình liên tục nhằm nâng cao năng lực tổ chức trong việc phát triển phần mềm.

Minh họa và Ví dụ Thực tế



Hình 10.1 - So sánh các mô hình vòng đời phần mềm: Waterfall, Iterative/Agile, DevOps.



Hình 10.2 - Các mức trưởng thành trong mô hình CMMI, từ Initial đến Optimizing.

YAML - Định nghĩa cổng chất lượng theo giai đoạn quy trình (process gate)

Ví dụ 10.1 - Một thực hành cải tiến quy trình: mã hóa các tiêu chí "cổng" (gate criteria) phải đạt trước khi một thay đổi được phép đi qua từng giai đoạn của quy trình, giúp quy trình trở nên có thể đo lường và lặp lại (yếu tố cốt lõi để đạt CMMI mức 3+):

```
# process-gates.yaml - cấu hình đọc bởi pipeline nội bộ
gates:
  - stage: code_review
    require:
      - min_approvals: 2
      - no_unresolved_comments: true
  - stage: pre_merge
    require:
      - test_coverage_min_percent: 80
      - static_analysis_critical_issues: 0
  - stage: pre_release
    require:
      - security_scan_passed: true
      - performance_regression_percent_max: 5
      - rollback_plan_documented: true
```

Bài tập Thực hành

Bài 1. Với một dự án bạn biết rõ, xác định xem tổ chức đang ở mức trưởng thành CMMI nào (1-5) dựa trên mô tả ở Hình 10.2, và đề xuất 2 hành động cụ thể để tiến lên mức kế tiếp.

Bài 2. So sánh ưu/nhược điểm của mô hình Waterfall và Agile cho một dự án xây dựng phần mềm điều khiển thiết bị y tế (an toàn nghiêm ngặt) - mô hình nào phù hợp hơn và vì sao?

Bài 3. Thiết kế 3 tiêu chí 'cổng chất lượng' bạn sẽ áp dụng trước khi cho phép merge code vào nhánh chính của một dự án thực tế.

CHƯƠNG 11

Mô hình và Phương pháp KỸ THUẬT PHẦN MỀM

Software Engineering Models and Methods

Lĩnh vực này đề cập đến các mô hình hóa (modeling) và phương pháp luận (methods) được dùng xuyên suốt các giai đoạn của KỸ THUẬT PHẦN MỀM để biểu diễn, phân tích và thiết kế hệ thống phần mềm ở mức trừu tượng.

Góc nhìn chuyên sâu: việc lựa chọn mức độ hình thức hóa (formality) khi mô hình hóa là một đánh đổi thực tế - mô hình hình thức đầy đủ (formal methods, dùng logic toán học để chứng minh tính đúng đắn) mang lại độ tin cậy cao nhất nhưng tốn thời gian và đòi hỏi chuyên môn hiếm, nên chỉ hợp lý cho hệ thống có rủi ro cực cao (điều khiển hàng không, hạt nhân). Với phần lớn phần mềm thương mại, các mô hình bán hình thức (semi-formal) như UML hoặc máy trạng thái tường minh (Ví dụ 11.1) mang lại giá trị tốt nhất trên mỗi đơn vị công sức bỏ ra - đủ chặt chẽ để phát hiện lỗi thiết kế sớm, nhưng không quá nặng nề để làm chậm tốc độ phát triển.

1. Mô hình hóa (Modeling)

Là việc xây dựng biểu diễn trừu tượng của hệ thống nhằm hiểu, giao tiếp và phân tích các khía cạnh khác nhau của phần mềm mà không cần đi vào toàn bộ chi tiết triển khai.

2. Các Loại Mô hình

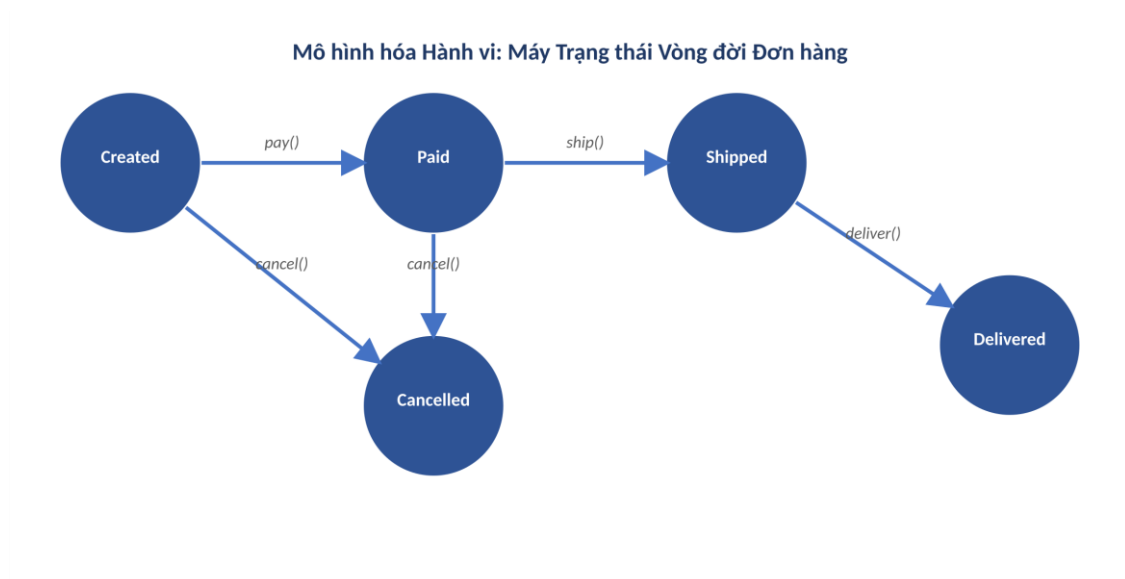
Bao gồm mô hình thông tin/dữ liệu (mô tả cấu trúc dữ liệu), mô hình hành vi (mô tả trạng thái và tương tác theo thời gian), và mô hình cấu trúc (mô tả thành phần và mối quan hệ tĩnh của hệ thống).

3. Phân tích Mô hình

Bao gồm các kỹ thuật kiểm tra tính nhất quán, đầy đủ và chính xác của mô hình, cũng như mô phỏng để dự đoán hành vi hệ thống trước khi triển khai thực tế.

4. Phương pháp KỸ THUẬT PHẦN MỀM

Bao gồm các phương pháp luận hệ thống hóa như phát triển hướng đối tượng, phát triển hướng mô hình (model-driven development), phương pháp hình thức (formal methods) và các phương pháp Agile nhấn mạnh sự lặp lại và phản hồi nhanh.

Minh họa và Ví dụ Thực tế

Hình 11.1 - Mô hình hóa hành vi bằng máy trạng thái hữu hạn: vòng đời một đơn hàng.

Python - Cài đặt máy trạng thái hữu hạn (FSM) tường minh, chống chuyển trạng thái sai

Ví dụ 11.1 - Mô hình hóa vòng đời đối tượng nghiệp vụ bằng FSM tường minh (dùng thư viện `transitions`), đảm bảo không thể chuyển sang trạng thái không hợp lệ - kỹ thuật mô hình hóa hành vi cốt lõi của Chương 11:

```
from transitions import Machine

class Order:
    states = ['created', 'paid', 'shipped', 'delivered', 'cancelled']

    def __init__(self):
        self.machine = Machine(
            model=self, states=Order.states, initial='created'
        )
        self.machine.add_transition('pay', 'created', 'paid')
        self.machine.add_transition('ship', 'paid', 'shipped')
        self.machine.add_transition('deliver', 'shipped', 'delivered')
        self.machine.add_transition('cancel', ['created', 'paid'], 'cancelled')

order = Order()
order.pay()    # created -> paid
order.ship()   # paid -> shipped
try:
    order.cancel() # LỖI: không thể hủy đơn đã giao (shipped không có
transition 'cancel')
except Exception as e:
    print(f"Chuyển trạng thái không hợp lệ: {e}")
```

Bài tập Thực hành

Bài 1. Vẽ một sơ đồ trạng thái (state diagram) cho vòng đời của một 'phiên đăng nhập' (session) người dùng: chưa đăng nhập → đã đăng nhập → hết hạn/đăng xuất.

Bài 2. Cài đặt bằng code (ngôn ngữ tùy chọn) máy trạng thái cho ví dụ ở câu 1, đảm bảo không thể chuyển từ trạng thái 'chưa đăng nhập' thẳng sang 'hết hạn'.

Bài 3. So sánh việc dùng FSM tường minh với việc dùng nhiều cờ boolean (is_paid, is_shipped, is_cancelled...) để quản lý trạng thái đơn hàng - nêu ít nhất 2 rủi ro của cách tiếp cận dùng cờ boolean.

CHƯƠNG 12

Chất lượng Phần mềm

Software Quality

Chất lượng phần mềm là mức độ mà một hệ thống, thành phần hoặc quy trình đáp ứng các yêu cầu đã xác định và nhu cầu, kỳ vọng của người dùng và khách hàng.

Góc nhìn chuyên sâu: chất lượng phần mềm luôn là một sự đánh đổi có chủ đích, không phải một giá trị tuyệt đối cần tối đa hóa ở mọi khía cạnh cùng lúc - một sản phẩm MVP (Minimum Viable Product) của startup ưu tiên tốc độ ra thị trường có thể chấp nhận nợ kỹ thuật tạm thời, trong khi phần mềm điều khiển thiết bị y tế không thể thỏa hiệp về độ tin cậy dù chậm hơn. Vai trò của kỹ sư chất lượng không phải là đòi hỏi "chất lượng cao nhất có thể" một cách máy móc, mà là giúp tổ chức đưa ra quyết định đánh đổi có ý thức - biết rõ mình đang hy sinh đặc tính nào để đổi lấy đặc tính nào, và mức độ rủi ro đi kèm.

1. Nền tảng Chất lượng Phần mềm

Bao gồm các mô hình chất lượng chuẩn hóa như ISO/IEC 25010 với các đặc tính chất lượng (functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, portability), làm cơ sở để định nghĩa và đo lường chất lượng một cách hệ thống.

2. Quy trình Quản lý Chất lượng Phần mềm

Bao gồm việc lập kế hoạch chất lượng, xác định chỉ tiêu chất lượng và thiết lập cơ chế giám sát việc đạt được các chỉ tiêu đó trong suốt vòng đời dự án.

3. Quy trình Bảo đảm Chất lượng Phần mềm (SQA)

Bao gồm các hoạt động rà soát, kiểm định (audit), và kiểm chứng - xác nhận (verification and validation) nhằm bảo đảm quy trình và sản phẩm tuân thủ các tiêu chuẩn chất lượng đã thiết lập.

4. Công cụ Chất lượng Phần mềm

Bao gồm công cụ phân tích mã tĩnh, công cụ đo lường độ phức tạp, công cụ theo dõi lỗi và các nền tảng đánh giá chất lượng liên tục tích hợp trong pipeline phát triển.

Minh họa và Ví dụ Thực tế

Các Đặc tính Chất lượng - ISO/IEC 25010



Hình 12.1 - Tám đặc tính chất lượng phần mềm theo ISO/IEC 25010.

YAML - SonarQube Quality Gate: kiểm soát chất lượng tự động trong pipeline

Ví dụ 12.1 - Cấu hình cổng chất lượng (Quality Gate) chặn merge nếu không đạt ngưỡng, gắn liền quy trình bảo đảm chất lượng vào CI/CD:

```
# sonar-project.properties
sonar.projectKey=order-service
sonar.qualitygate.wait=true

# Ngưỡng chất lượng bắt buộc để merge:
# - Độ bao phủ kiểm thử (coverage) mới >= 80%
# - Không có lỗi hỏng bảo mật mức Critical/Blocker
# - Tỷ lệ trùng lặp mã nguồn (duplication) < 3%
# - Độ phức tạp cyclomatic trung bình mỗi hàm <= 10
# - Không có 'code smell' mức độ nghiêm trọng cao
```

Python - Tính điểm chất lượng tổng hợp từ nhiều chỉ số (weighted quality score)

Ví dụ 12.2 - Minh họa cách một tổ chức lượng hóa chất lượng phần mềm thành một điểm số tổng hợp duy nhất để theo dõi xu hướng theo thời gian:

```
WEIGHTS = {
    'test_coverage': 0.30,
    'code_duplication_inverse': 0.20,
    'security_score': 0.30,
    'maintainability_index': 0.20,
}

def quality_score(metrics: dict[str, float]) -> float:
    """Mỗi metric đã được chuẩn hóa về thang 0-100 trước khi truyền vào."""
    return sum(metrics[key] * weight for key, weight in WEIGHTS.items())

current = {
    'test_coverage': 82,
    'code_duplication_inverse': 91, # 100 - % trùng lặp
    'security_score': 95,
    'maintainability_index': 74,
}

print(f"Điểm chất lượng tổng hợp: {quality_score(current):.1f}/100")
```

Bài tập Thực hành

Bài 1. Chọn một dự án cá nhân, chạy công cụ phân tích tĩnh miễn phí (SonarQube Community, CodeClimate, hoặc ESLint) và ghi lại 3 vấn đề chất lượng lớn nhất được phát hiện.

Bài 2. Với 8 đặc tính ISO/IEC 25010 ở Hình 12.1, chọn 3 đặc tính bạn cho là quan trọng nhất đối với một ứng dụng ngân hàng di động, và giải thích vì sao.

Bài 3. Thiết kế công thức tính điểm chất lượng tổng hợp (như Ví dụ 12.2) phù hợp hơn cho một hệ thống game trực tuyến.

CHƯƠNG 13

An toàn Phần mềm

Software Security

Đây là lĩnh vực tri thức mới thứ ba trong SWEBOK V4.0, phản ánh nhận thức ngày càng sâu sắc rằng bảo mật phải được tích hợp xuyên suốt vòng đời phát triển phần mềm (Security by Design) thay vì chỉ được xử lý như một lớp bổ sung sau khi hệ thống đã hoàn thiện.

Góc nhìn chuyên sâu: nguyên tắc "phòng thủ theo chiều sâu" (defense in depth) là tư duy cốt lõi của kỹ nghệ an toàn hiện đại - không có một biện pháp bảo mật đơn lẻ nào là hoàn hảo, nên hệ thống an toàn cần nhiều lớp bảo vệ độc lập (xác thực mạnh, mã hóa dữ liệu, giám sát bất thường, phân quyền tối thiểu) sao cho nếu một lớp bị vượt qua, các lớp khác vẫn ngăn chặn được thiệt hại. Với sự trỗi dậy của các cuộc tấn công chuỗi cung ứng phần mềm (supply chain attacks, ví dụ khai thác lỗ hổng trong thư viện phụ thuộc phổ biến) giai đoạn 2023-2026, việc quét và giám sát dependency (như SCA ở Hình 13.1) đã trở thành yêu cầu bắt buộc chứ không còn là lựa chọn nâng cao, đặc biệt với hệ sinh thái npm/PyPI có hàng triệu gói phụ thuộc lẫn nhau.

1. Nền tảng An toàn Phần mềm

Bao gồm các nguyên lý bảo mật cốt lõi: tính bí mật (confidentiality), tính toàn vẹn (integrity), tính sẵn sàng (availability) - thường gọi tắt là CIA triad - cùng với các khái niệm về mối đe dọa (threat), lỗ hổng (vulnerability) và rủi ro bảo mật (risk).

2. Quản lý và Tổ chức An toàn

Bao gồm việc thiết lập chính sách bảo mật tổ chức, phân định vai trò trách nhiệm về an toàn thông tin, và xây dựng văn hóa bảo mật trong toàn bộ tổ chức phát triển phần mềm.

3. Kỹ nghệ và Quy trình An toàn Phần mềm

Bao gồm việc tích hợp các hoạt động bảo mật vào từng giai đoạn vòng đời phần mềm - mô hình hóa mối đe dọa (threat modeling) khi thiết kế, rà soát mã nguồn bảo mật khi xây dựng, kiểm thử thâm nhập (penetration testing) trước khi phát hành.

4. Kỹ nghệ An toàn cho Hệ thống Phần mềm

Bao gồm các kỹ thuật thiết kế an toàn cụ thể như xác thực và phân quyền (authentication/authorization), mã hóa dữ liệu, quản lý bí mật (secrets management) và các mẫu kiến trúc bảo mật (zero trust, defense in depth).

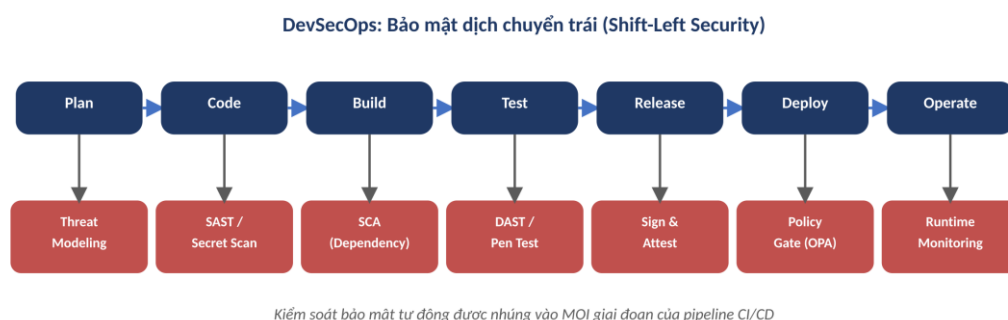
5. Công cụ An toàn Phần mềm

Bao gồm công cụ phân tích tĩnh bảo mật (SAST), phân tích động bảo mật (DAST), công cụ quét lỗ hổng phụ thuộc (dependency scanning) và các nền tảng quản lý bảo mật tích hợp trong DevSecOps.

6. An toàn Phần mềm theo Miền Ứng dụng cụ thể

Đề cập đến các yêu cầu bảo mật đặc thù theo lĩnh vực, ví dụ hệ thống tài chính, y tế, hạ tầng trọng yếu và hệ thống nhúng/IoT, mỗi lĩnh vực có bộ quy định và mối đe dọa đặc thù riêng.

Minh họa và Ví dụ Thực tế



Hình 13.1 - Bảo mật dịch chuyển trái (shift-left): các cổng kiểm soát bảo mật gắn vào từng giai đoạn pipeline.

TypeScript - So sánh mã dễ bị tấn công SQL Injection và mã an toàn (parameterized query)

Ví dụ 13.1 - Minh họa nguyên tắc kỹ nghệ an toàn cơ bản nhất: không bao giờ nối chuỗi trực tiếp vào câu truy vấn:

```
// KHÔNG AN TOÀN - dễ bị SQL Injection
async function findUserUnsafe(email: string) {
  return db.query(
    `SELECT * FROM users WHERE email = '${email}'`
  );
}

// AN TOÀN - dùng tham số hóa (parameterized query)
async function findUserSafe(email: string) {
  return db.query(
    'SELECT id, name, email FROM users WHERE email = $1',
    [email]
  );
}
```

YAML - GitHub Actions: quét bảo mật tự động (SAST + kiểm tra dependency)

Ví dụ 13.2 - Pipeline DevSecOps tối thiểu: quét mã tĩnh (CodeQL) và quét lỗi hỏng thư viện phụ thuộc (dependency scanning) chạy tự động trên mỗi pull request:

```
name: Security Scan
on: [pull_request]

jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with: { languages: javascript-typescript }
      - uses: github/codeql-action/analyze@v3

  dependency-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Quét lỗi hỏng dependency
        run: npx audit-ci --critical --high
      - name: Quét bí mật bị lộ (secrets)
        uses: trufflesecurity/trufflehog@main
```

TypeScript - Xác thực & phân quyền: middleware kiểm tra JWT và vai trò (RBAC)

Ví dụ 13.3 - Cài đặt kiểm soát truy cập dựa trên vai trò (Role-Based Access Control) - một trong những kỹ thuật kỹ nghệ an toàn nền tảng nhất cho hệ thống nhiều người dùng:

```
import jwt from 'jsonwebtoken';

type Role = 'customer' | 'support_agent' | 'admin';

function requireRole(...allowedRoles: Role[]) {
  return (req, res, next) => {
    const token = req.headers.authorization?.replace('Bearer ', '');
    if (!token) return res.status(401).json({ error: 'Thiếu token' });

    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET!) as { role: Role };
      if (!allowedRoles.includes(payload.role)) {
        return res.status(403).json({ error: 'Không đủ quyền truy cập' });
      }
      next();
    } catch {
      return res.status(401).json({ error: 'Token không hợp lệ hoặc hết hạn' });
    }
  };
}

// Chỉ admin và support_agent mới được xem toàn bộ đơn hàng của khách khác
app.get('/admin/orders/:userId', requireRole('admin', 'support_agent'),
getOrders);
```

Bài tập Thực hành

Bài 1. Tìm một đoạn code cũ của bạn có thao tác truy vấn cơ sở dữ liệu. Kiểm tra xem có bị lỗi SQL Injection tiềm ẩn không, và sửa lại bằng parameterized query nếu cần.

Bài 2. Thiết lập một GitHub Action đơn giản chạy `npm audit` hoặc công cụ tương đương cho một repo của bạn, để tự động phát hiện dependency có lỗi hỏng.

Bài 3. Thiết kế mô hình phân quyền (RBAC) cho một ứng dụng quản lý bệnh viện với ít nhất 3 vai trò, liệt kê quyền truy cập của mỗi vai trò.

CHƯƠNG 14

Thực hành Nghề nghiệp KỸ THUẬT PHẦN MỀM

Software Engineering Professional Practice

Lĩnh vực này đề cập đến các kỹ năng và trách nhiệm mang tính chuyên môn, đạo đức và xã hội mà một kỹ sư phần mềm cần có ngoài kiến thức kỹ thuật thuần túy.

Góc nhìn chuyên sâu: kỹ năng "mềm" như giao tiếp và làm việc nhóm thường bị đánh giá thấp hơn kỹ năng kỹ thuật thuần túy trong đào tạo kỹ sư phần mềm, nhưng nghiên cứu và khảo sát ngành liên tục cho thấy nguyên nhân hàng đầu khiến dự án phần mềm thất bại là vấn đề giao tiếp và tổ chức, không phải vấn đề kỹ thuật. Một kỹ sư xuất sắc về mặt kỹ thuật nhưng không thể giải thích rõ ràng quyết định thiết kế, không lắng nghe phản hồi, hoặc né tránh xung đột mang tính xây dựng thường tạo ra rủi ro lớn hơn cho dự án so với một kỹ sư trung bình nhưng giao tiếp tốt. Bộ Quy tắc Đạo đức ACM/IEEE-CS đặt trách nhiệm nghề nghiệp và xã hội ngang hàng với năng lực kỹ thuật, phản ánh đúng thực tế này.

1. Tính chuyên nghiệp (Professionalism)

Bao gồm việc tuân thủ các quy tắc đạo đức nghề nghiệp (như Bộ Quy tắc Đạo đức ACM/IEEE-CS), trách nhiệm pháp lý, bảo vệ quyền riêng tư và an toàn công cộng, cũng như cam kết học tập và phát triển nghề nghiệp liên tục.

2. Động lực Nhóm và Tâm lý học

Bao gồm hiểu biết về động lực làm việc nhóm, giải quyết xung đột, xây dựng văn hóa hợp tác và các yếu tố tâm lý ảnh hưởng đến hiệu suất của nhóm phát triển phần mềm.

3. Kỹ năng Giao tiếp

Bao gồm kỹ năng giao tiếp bằng văn bản và lời nói, thuyết trình, đàm phán và làm việc hiệu quả với các bên liên quan có nền tảng chuyên môn khác nhau.

Minh họa và Ví dụ Thực tế

Các Giai đoạn Phát triển Nhóm (Mô hình Tuckman)



Hình 14.1 - Các giai đoạn phát triển nhóm theo mô hình Tuckman: Forming → Storming → Norming → Performing → Adjourning.

Markdown - Mẫu RACI Matrix cho trách nhiệm trong một dự án phần mềm

Ví dụ 14.1 - Công cụ thực hành nghề nghiệp phổ biến: ma trận RACI (Responsible Accountable Consulted Informed) giúp làm rõ vai trò và tránh xung đột trách nhiệm trong nhóm:

Hoạt động	Dev Lead	Product Owner	QA	DevOps
Viết đặc tả yêu cầu	C	A/R	I	I
Thiết kế kiến trúc	A/R	C	I	C
Viết mã & unit test	R	I	I	I
Kiểm thử chấp nhận	C	A/R	R	I
Triển khai production	C	I	I	A/R

R = Responsible (thực hiện), A = Accountable (chịu trách nhiệm cuối cùng)
 # C = Consulted (được tham vấn), I = Informed (được thông báo)

Bài tập Thực hành

Bài 1. Nhớ lại một nhóm bạn từng làm việc cùng (học tập hoặc công việc). Xác định nhóm đó đang ở giai đoạn nào trong mô hình Tuckman và mô tả 1 dấu hiệu cụ thể chứng minh điều đó.

Bài 2. Soạn một ma trận RACI cho một dự án nhóm 4 người mà bạn đang/sẽ thực hiện, với ít nhất 5 hoạt động.

Bài 3. Viết một đoạn hội thoại giả định (3-4 câu) thể hiện cách xử lý xung đột mang tính xây dựng giữa hai thành viên nhóm bất đồng về giải pháp kỹ thuật.

CHƯƠNG 15

Kinh tế học KỸ THUẬT PHẦN MỀM

Software Engineering Economics

Lĩnh vực này áp dụng các nguyên lý kinh tế học và phân tích ra quyết định vào các quyết định kỹ thuật và quản lý trong KỸ THUẬT PHẦN MỀM, giúp cân bằng giữa chi phí, giá trị và rủi ro.

Góc nhìn chuyên sâu: một trong những sai lầm kinh tế phổ biến nhất trong KỸ THUẬT PHẦN MỀM là "ngụy biện chi phí chìm" (sunk cost fallacy) - tiếp tục đầu tư vào một giải pháp kỹ thuật đã tỏ ra sai lầm chỉ vì đã bỏ ra quá nhiều công sức, thay vì đánh giá khách quan chi phí và lợi ích từ thời điểm hiện tại trở đi. Các kỹ thuật ra quyết định kinh tế như NPV (Ví dụ 15.1) buộc người ra quyết định phải nhìn về tương lai (dòng tiền còn lại) thay vì quá khứ (đã đầu tư bao nhiêu) - đây là nguyên tắc kinh tế học nền tảng nhưng thường bị vi phạm trong thực tế tổ chức vì áp lực tâm lý và chính trị nội bộ.

1. Nền tảng Kinh tế học KỸ THUẬT PHẦN MỀM

Bao gồm các khái niệm cơ bản về giá trị tiền tệ theo thời gian, chi phí cơ hội, và cách áp dụng tư duy kinh tế vào các quyết định kỹ thuật phần mềm.

2. Quy trình Ra quyết định Kỹ thuật

Bao gồm việc xác định các phương án thay thế, đánh giá chúng dựa trên nhiều tiêu chí và lựa chọn phương án tối ưu trong điều kiện nguồn lực hạn chế.

3. Ra quyết định vì Lợi nhuận

Áp dụng cho các tổ chức thương mại, sử dụng các chỉ số như giá trị hiện tại ròng (NPV), tỷ suất hoàn vốn nội bộ (IRR) và thời gian hoàn vốn để đánh giá tính khả thi tài chính của dự án.

4. Ra quyết định Phi lợi nhuận

Áp dụng cho các tổ chức phi lợi nhuận hoặc khu vực công, nơi các tiêu chí quyết định không chỉ dựa trên lợi nhuận tài chính mà còn dựa trên giá trị xã hội và sứ mệnh tổ chức.

5. Ra quyết định theo Kinh tế Hiện tại

Đề cập đến các kỹ thuật so sánh phương án dựa trên chi phí và lợi ích ở thời điểm hiện tại, không cần chiết khấu dòng tiền phức tạp.

6. Ra quyết định Đa thuộc tính

Sử dụng các phương pháp phân tích đa tiêu chí (multi-criteria decision analysis) khi phải cân nhắc nhiều thuộc tính cạnh tranh nhau như chi phí, chất lượng và thời gian.

7. Nhận diện và Mô tả Tài sản Vô hình

Bao gồm việc đánh giá các tài sản khó định lượng như uy tín thương hiệu, tri thức tổ chức và giá trị chiến lược dài hạn của các quyết định kỹ nghệ.

8. Ước lượng (Estimation)

Bao gồm các kỹ thuật ước lượng chi phí và thời gian dự án như COCOMO, Function Point Analysis, ước lượng dựa trên chuyên gia (expert judgment) và ước lượng tương tự (analogy-based).

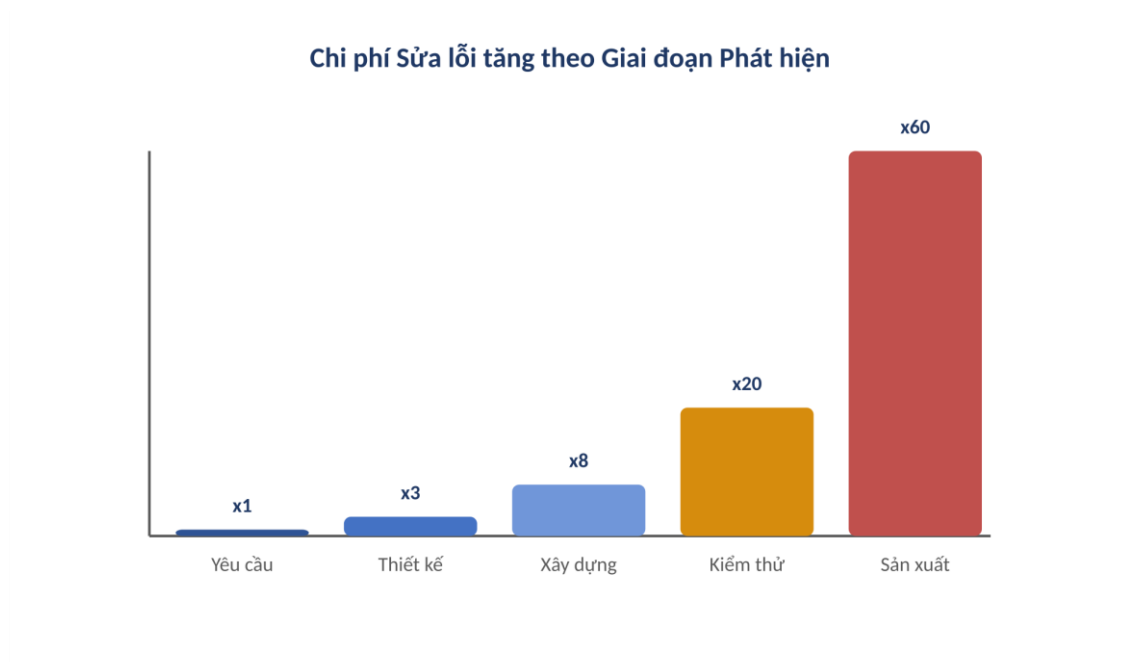
9. Cân nhắc Thực tiễn

Đề cập đến các thách thức thực tế trong ước lượng và ra quyết định kinh tế, như sự không chắc chắn cao trong giai đoạn đầu dự án và thiên kiến nhận thức của người ước lượng.

10. Khái niệm Liên quan

Bao gồm các khái niệm hỗ trợ như phân tích rủi ro tài chính, mô hình hóa chi phí vòng đời phần mềm và giá trị kinh doanh của chất lượng phần mềm.

Minh họa và Ví dụ Thực tế



Hình 15.1 - Chi phí sửa lỗi tăng theo cấp số nhân khi phát hiện muộn trong vòng đời phần mềm.

Python - Tính giá trị hiện tại ròng (NPV) để so sánh hai phương án đầu tư kỹ thuật

Ví dụ 15.1 - Ứng dụng kinh tế học KỸ THUẬT PHẦN MỀM: dùng NPV để so sánh giữa việc 'vá tạm' hệ thống cũ và 'viết lại' bằng kiến trúc mới, hỗ trợ ra quyết định dựa trên số liệu tài chính thay vì cảm tính:

```
def npv(cash_flows: list[float], discount_rate: float) -> float:
    return sum(
        cf / (1 + discount_rate) ** year
        for year, cf in enumerate(cash_flows)
    )

# Phương án A: Vá tạm hệ thống cũ - chi phí thấp ban đầu, tổn kém dần theo
# thời gian
patch_option = [-200_000_000, -150_000_000, -180_000_000, -220_000_000]

# Phương án B: Viết lại bằng kiến trúc mới - đầu tư lớn ban đầu, tiết kiệm về
# sau
rewrite_option = [-800_000_000, -50_000_000, -50_000_000, -50_000_000]

rate = 0.10 # chi phí vốn 10%/năm
print(f'NPV vá tạm: {npv(patch_option, rate):,.0f} VND')
print(f'NPV viết lại: {npv(rewrite_option, rate):,.0f} VND')
# NPV càng gần 0 (ít âm hơn) càng là phương án kinh tế hơn trong dài hạn
```

Bài tập Thực hành

Bài 1. Dùng công thức NPV ở Ví dụ 15.1, thử thay đổi discount_rate thành 5% và 20% - kết quả so sánh giữa 2 phương án có thay đổi không? Giải thích ý nghĩa.

Bài 2. Ước lượng chi phí (bằng COCOMO đơn giản hoặc ước lượng chuyên gia) cho một tính năng nhỏ bạn từng làm: số giờ công, chi phí nhân sự, và so sánh với thời gian thực tế đã tốn.

Bài 3. Áp dụng Hình 15.1: nếu một lỗi được phát hiện ở giai đoạn Kiểm thử tốn 20 đơn vị chi phí để sửa, hãy ước tính lỗi đó sẽ tốn bao nhiêu nếu phát hiện ở giai đoạn Sản xuất, và đề xuất 1 biện pháp phát hiện sớm hơn.

CHƯƠNG 16

Nền tảng Điện toán

Computing Foundations

Lĩnh vực này bao quát các kiến thức nền tảng về khoa học máy tính mà mọi kỹ sư phần mềm cần nắm vững để xây dựng phần mềm hiệu quả và đúng đắn.

Góc nhìn chuyên sâu: việc tích hợp các thành phần AI/ML vào hệ thống phần mềm mang đến một lớp thách thức kỹ nghệ mới mà nền tảng điện toán truyền thống chưa từng đối mặt - đầu ra của mô hình AI mang tính xác suất (non-deterministic), khác biệt căn bản với hàm số truyền thống luôn trả về cùng một kết quả với cùng đầu vào. Điều này đòi hỏi kỹ sư phần mềm năm 2026 phải có tư duy mới: không tin tưởng tuyệt đối vào đầu ra của mô hình AI (luôn validate bằng schema như Ví dụ 16.1), thiết kế cơ chế fallback khi mô hình trả lời sai hoặc không khả dụng, và giám sát "model drift" (mô hình dần kém chính xác theo thời gian do dữ liệu thực tế thay đổi) - những khái niệm gần như không tồn tại trong KỸ THUẬT PHẦN MỀM truyền thống.

1. Khái niệm Cơ bản của Hệ thống hoặc Giải pháp

Bao gồm tư duy hệ thống, hiểu biết về đầu vào-xử lý-đầu ra và mối quan hệ giữa các thành phần trong một giải pháp phần mềm tổng thể.

2. Kiến trúc và Tổ chức Máy tính

Bao gồm hiểu biết về cấu trúc phần cứng máy tính, bộ xử lý, bộ nhớ và cách chúng ảnh hưởng đến hiệu năng phần mềm.

3. Cấu trúc Dữ liệu và Giải thuật

Bao gồm các cấu trúc dữ liệu nền tảng (mảng, danh sách liên kết, cây, đồ thị, bảng băm) và các giải thuật sắp xếp, tìm kiếm, phân tích độ phức tạp thuật toán (Big-O notation).

4. Nền tảng Lập trình và Ngôn ngữ Lập trình

Bao gồm các mô hình lập trình (thủ tục, hướng đối tượng, hướng hàm), cú pháp và ngữ nghĩa ngôn ngữ lập trình, và các nguyên lý thiết kế ngôn ngữ.

5. Hệ điều hành

Bao gồm quản lý tiến trình, quản lý bộ nhớ, hệ thống tệp và các cơ chế đồng bộ hóa mà phần mềm ứng dụng phải tương tác.

6. Quản lý Cơ sở Dữ liệu

Bao gồm mô hình dữ liệu quan hệ và phi quan hệ, ngôn ngữ truy vấn (SQL), thiết kế cơ sở dữ liệu và các nguyên lý giao dịch (ACID).

7. Mạng máy tính và Truyền thông

Bao gồm các mô hình giao thức mạng (TCP/IP), kiến trúc client-server, các giao thức web (HTTP/HTTPS) và nguyên lý truyền thông phân tán.

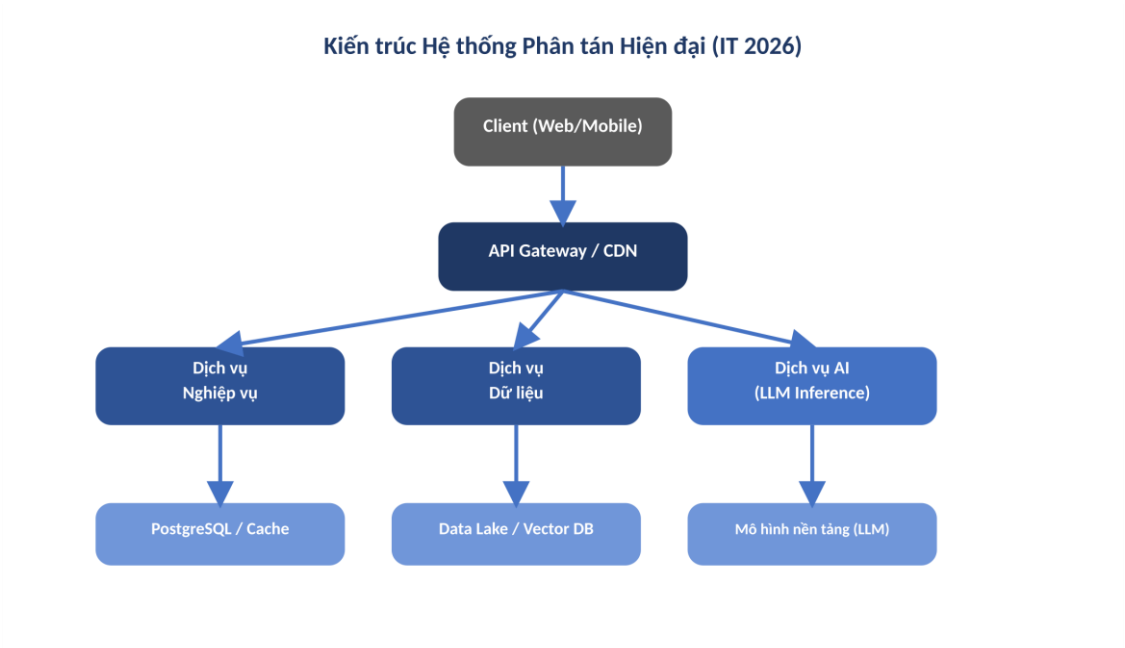
8. Yếu tố Con người của Người dùng và Người phát triển

Bao gồm nguyên lý thiết kế giao diện người dùng (UI/UX), khả năng sử dụng (usability), khả năng tiếp cận (accessibility) và công thái học nhận thức trong tương tác người-máy.

9. Trí tuệ Nhân tạo và Học máy

Là nội dung được mở rộng đáng kể trong V4.0, bao gồm các khái niệm cơ bản về học máy, mạng nơ-ron, mô hình ngôn ngữ lớn và cách tích hợp các thành phần AI/ML vào hệ thống phần mềm một cách có trách nhiệm.

Minh họa và Ví dụ Thực tế



Hình 16.1 - Kiến trúc hệ thống phân tán hiện đại có tích hợp dịch vụ AI/LLM (tiêu biểu năm 2026).

Python - Tích hợp mô hình ngôn ngữ lớn (LLM) vào dịch vụ backend theo mẫu Adapter

Ví dụ 16.1 - Nền tảng điện toán 2026: tích hợp một dịch vụ AI (LLM) vào hệ thống backend một cách có kiểm soát - dùng schema đầu ra chặt chẽ (structured output) thay vì tin tưởng mù quáng vào văn bản tự do:

```
from pydantic import BaseModel
import anthropic

client = anthropic.Anthropic()

class TicketClassification(BaseModel):
    category: str      # 'billing' | 'technical' | 'account'
    priority: str      # 'low' | 'medium' | 'high' | 'urgent'
    summary: str

def classify_support_ticket(ticket_text: str) -> TicketClassification:
    response = client.messages.create(
        model="claude-sonnet-5",
        max_tokens=300,
        messages=[{
            "role": "user",
            "content": (
                "Phân loại yêu cầu hỗ trợ sau và trả về JSON đúng "
                f"schema TicketClassification:\n\n{ticket_text}"
            )
        }],
    )
    # Trong hệ thống thực tế: xác thực JSON trả về bằng Pydantic
    # trước khi đưa vào luồng nghiệp vụ, không tin dữ liệu mô hình 100%.
    return TicketClassification.model_validate_json(response.content[0].text)
```

SQL - Truy vấn tối ưu với chỉ mục (index) trên cơ sở dữ liệu quan hệ

Ví dụ 16.2 - Nền tảng điện toán cốt lõi: minh họa cách một chỉ mục (index) hợp lý biến đổi độ phức tạp truy vấn từ $O(n)$ quét toàn bảng thành $O(\log n)$ tra cứu theo cây B-Tree:

```
-- Không có chỉ mục: cơ sở dữ liệu phải quét toàn bộ bảng orders (chậm khi
bảng lớn)
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_email = 'khach@example.com';
-- => Seq Scan on orders (cost=0.00..18342.00 rows=1)

-- Thêm chỉ mục trên cột thường xuyên được truy vấn
CREATE INDEX idx_orders_customer_email ON orders (customer_email);

-- Truy vấn lại: PostgreSQL giờ dùng Index Scan, nhanh hơn nhiều bậc
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_email = 'khach@example.com';
-- => Index Scan using idx_orders_customer_email (cost=0.42..8.44 rows=1)
```

Bài tập Thực hành

Bài 1. Viết một hàm gọi API của một mô hình AI (bất kỳ, có thể dùng API miễn phí) để phân loại văn bản thành 2-3 nhãn, và validate kết quả trả về bằng một schema (Pydantic hoặc Zod).

Bài 2. Trên một cơ sở dữ liệu bạn có quyền truy cập (hoặc SQLite cục bộ), thử tạo một bảng với 10.000 dòng dữ liệu giả, đo thời gian truy vấn trước và sau khi thêm chỉ mục.

Bài 3. Vẽ sơ đồ kiến trúc hệ thống cho một ứng dụng chatbot hỗ trợ khách hàng có tích hợp AI, xác định rõ: nơi lưu trữ lịch sử hội thoại, nơi gọi mô hình AI, và cơ chế giới hạn tốc độ.

CHƯƠNG 17

Nền tảng Toán học

Mathematical Foundations

Toán học rời rạc cung cấp ngôn ngữ hình thức và công cụ tư duy chặt chẽ cần thiết cho việc đặc tả, thiết kế và kiểm chứng phần mềm một cách chính xác.

Góc nhìn chuyên sâu: nhiều sinh viên và kỹ sư trẻ coi toán học rời rạc là môn học hàn lâm xa rời thực tế, nhưng thực chất mọi hệ thống phần mềm hiện đại đều vận hành dựa trên các cấu trúc toán học này một cách âm thầm - từ máy trạng thái hữu hạn điều khiển luồng giao diện người dùng, đồ thị định tuyến mạng và gợi ý sản phẩm, đến lý thuyết số nền tảng của mọi giao dịch HTTPS bạn thực hiện mỗi ngày (Ví dụ 17.1). Sự khác biệt giữa một kỹ sư giỏi và một kỹ sư xuất sắc thường nằm ở khả năng nhận diện khi nào một bài toán thực tế có thể được ánh xạ về một cấu trúc toán học đã được nghiên cứu kỹ, thay vì phát minh lại giải pháp từ đầu một cách kém hiệu quả.

1. Logic Cơ bản

Bao gồm logic mệnh đề và logic vị từ, là nền tảng cho việc lập luận hình thức và kiểm chứng chương trình.

2. Kỹ thuật Chứng minh

Bao gồm chứng minh trực tiếp, chứng minh phản chứng và chứng minh quy nạp - các công cụ thiết yếu để xác minh tính đúng đắn của giải thuật.

3. Tập hợp, Quan hệ, Hàm số

Là các cấu trúc toán học nền tảng dùng để mô hình hóa dữ liệu và mối quan hệ giữa các thực thể trong hệ thống phần mềm.

4. Đồ thị và Cây

Là công cụ mô hình hóa quan trọng cho nhiều bài toán trong khoa học máy tính, từ cấu trúc dữ liệu đến biểu diễn mạng và luồng điều khiển chương trình.

5. Máy trạng thái hữu hạn

Được dùng rộng rãi để mô hình hóa hành vi hệ thống, đặc biệt trong thiết kế giao thức, trình biên dịch và hệ thống nhúng.

6. Văn phạm (Grammar)

Là nền tảng của lý thuyết ngôn ngữ hình thức, được ứng dụng trong thiết kế ngôn ngữ lập trình, trình biên dịch và xử lý ngôn ngữ tự nhiên.

7. Lý thuyết Số

Cung cấp nền tảng cho các thuật toán mật mã học và bảo mật, như các phép toán số học modulo dùng trong mã hóa khóa công khai.

8. Cơ bản về Đếm

Bao gồm các nguyên lý tổ hợp và hoán vị, hỗ trợ phân tích độ phức tạp thuật toán và xác suất.

9. Xác suất Rời rạc

Là nền tảng cho phân tích thuật toán ngẫu nhiên hóa, học máy và đánh giá độ tin cậy hệ thống.

10. Độ chính xác, Độ đúng đắn số học và Sai số

Đề cập đến các vấn đề biểu diễn số trong máy tính (dấu phẩy động), sai số làm tròn và ảnh hưởng của chúng đến độ tin cậy của tính toán.

11. Cấu trúc Đại số

Bao gồm các cấu trúc như nhóm, vành, trường - nền tảng lý thuyết cho mật mã học hiện đại và một số thuật toán chuyên biệt.

12. Giải tích Kỹ thuật

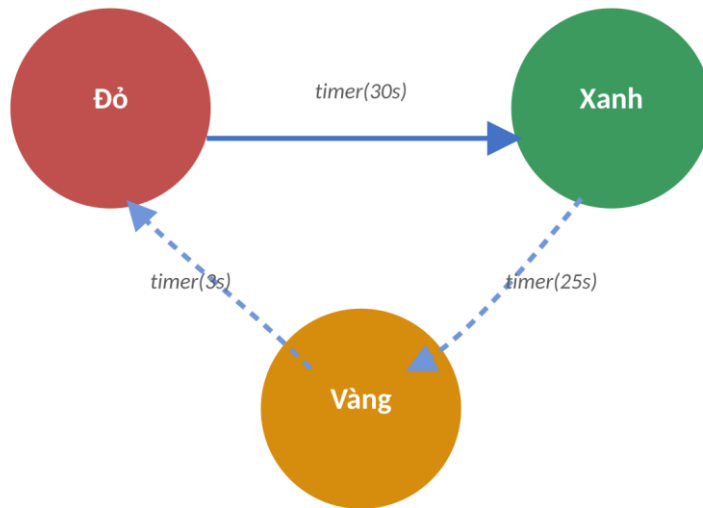
Bao gồm các khái niệm giải tích ứng dụng cần thiết cho một số lĩnh vực KỸ THUẬT PHẦN MỀM chuyên sâu như xử lý tín hiệu, đồ họa máy tính và mô phỏng.

13. Tiến bộ Mới

Đề cập đến các hướng toán học mới nổi có liên quan đến KỸ THUẬT PHẦN MỀM hiện đại, ví dụ đại số tuyến tính ứng dụng trong học sâu và lý thuyết tối ưu hóa.

Minh họa và Ví dụ Thực tế

Ví dụ Máy Trạng thái Hữu hạn: Đèn Giao thông



Hình 17.1 - Ví dụ cụ thể về máy trạng thái hữu hạn: hệ thống đèn giao thông ba trạng thái.

Python - Ứng dụng lý thuyết số vào mật mã học: thuật toán lũy thừa modulo nhanh

Ví dụ 17.1 - Minh họa ứng dụng thực tế của Lý thuyết Số (Chương 17, mục 7): phép lũy thừa modulo là phép toán lõi trong mã hóa khóa công khai RSA - được cài đặt hiệu quả bằng kỹ thuật "bình phương lặp" (fast exponentiation), độ phức tạp $O(\log n)$ thay vì $O(n)$:

```
def mod_pow(base: int, exponent: int, modulus: int) -> int:
    """Tính (base^exponent) mod modulus theo O(log exponent)."""
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent % 2 == 1:      # bit hiện tại là 1
            result = (result * base) % modulus
        exponent //= 2
        base = (base * base) % modulus # bình phương liên tiếp
    return result

# Trong RSA: mã hóa là  $c = m^e \bmod n$ , giải mã là  $m = c^d \bmod n$ 
public_exponent, modulus = 65537, 3233
message = 65
ciphertext = mod_pow(message, public_exponent, modulus)
print(f"Bản mã: {ciphertext}")

# So sánh: base**exponent (Python thuần) sẽ tạo ra số NGUYÊN KHỔNG LỒ
# và cực chậm với exponent lớn (hàng trăm chữ số như RSA thực tế);
# mod_pow() không bao giờ giữ số lớn hơn modulus.
```

Bài tập Thực hành

Bài 1. Vẽ sơ đồ máy trạng thái hữu hạn cho một máy bán nước tự động đơn giản: nhận tiền, chọn sản phẩm, trả hàng, trả lại tiền thừa.

Bài 2. Cài đặt hàm mod_pow ở Ví dụ 17.1 bằng ngôn ngữ khác (JavaScript/Java/C++), so sánh thời gian chạy với cách tính $base^{**}exponent \% modulus$ thông thường khi $exponent = 10.000$.

Bài 3. Chứng minh bằng quy nạp toán học rằng tổng $1 + 2 + \dots + n = n(n+1)/2$ với mọi số nguyên dương n - kỹ thuật chứng minh nền tảng của Chương 17, mục 2.

CHƯƠNG 18

Nền tảng Kỹ nghệ

Engineering Foundations

Lĩnh vực tri thức cuối cùng đặt KỸ THUẬT PHẦN MỀM trong bối cảnh rộng hơn của ngành kỹ nghệ nói chung, nhấn mạnh các nguyên lý kỹ thuật phổ quát mà mọi ngành kỹ nghệ - không riêng phần mềm - đều chia sẻ.

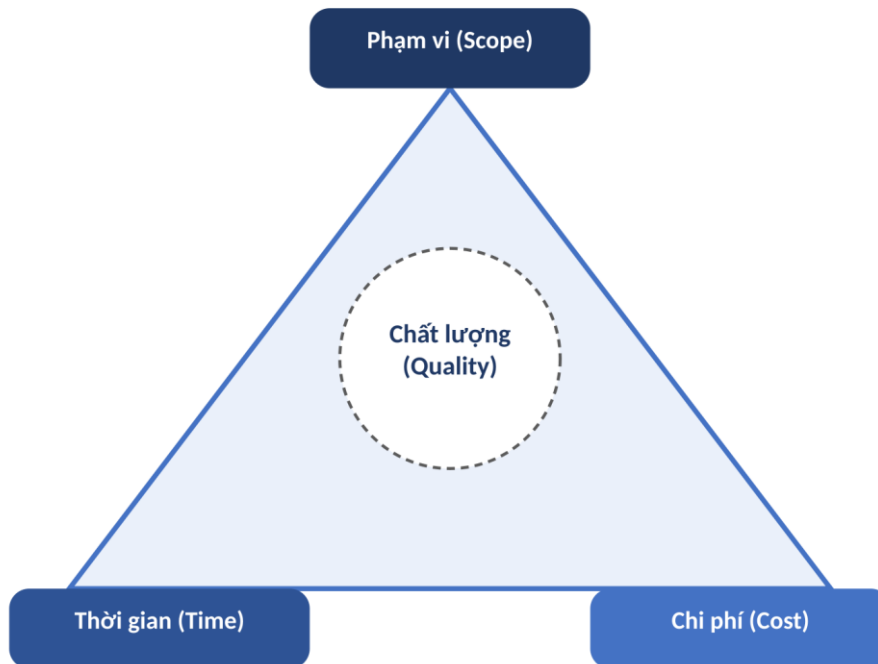
Góc nhìn chuyên sâu: sự khác biệt căn bản giữa KỸ THUẬT PHẦN MỀM và các ngành kỹ nghệ truyền thống (cơ khí, xây dựng, điện) nằm ở bản chất vô hình và cực kỳ dễ thay đổi của phần mềm - một cây cầu không thể "refactor" sau khi xây xong, nhưng phần mềm có thể được viết lại nhiều lần trong vòng đời của nó. Điều này vừa là lợi thế (linh hoạt thích nghi nhanh) vừa là cạm bẫy (dễ dẫn đến thiết kế tùy tiện, thiếu kỷ luật kỹ thuật vì "sửa sau cũng được"). Các nguyên lý kỹ nghệ nền tảng - tư duy hệ thống, quản lý đánh đổi có ý thức, trách nhiệm với an toàn công cộng - vẫn áp dụng đầy đủ cho phần mềm, nhưng đòi hỏi kỹ sư phần mềm phải chủ động tự đặt ra kỷ luật kỹ thuật cho chính mình, thay vì dựa vào các ràng buộc vật lý tự nhiên như trong kỹ nghệ truyền thống.

Tư duy và Phương pháp Kỹ nghệ

Bao gồm các nguyên lý chung của tư duy kỹ nghệ: giải quyết vấn đề có hệ thống, tối ưu hóa dưới các ràng buộc thực tế (chi phí, thời gian, nguồn lực), quản lý sự đánh đổi (trade-off) giữa các mục tiêu cạnh tranh, và trách nhiệm nghề nghiệp đối với an toàn và lợi ích công cộng - những nguyên lý mà KỸ THUẬT PHẦN MỀM kế thừa từ truyền thống kỹ nghệ nói chung (cơ khí, điện, xây dựng) và áp dụng vào đặc thù của sản phẩm phần mềm vốn vô hình và dễ thay đổi.

Minh họa và Ví dụ Thực tế

Tam giác Đánh đổi trong Kỹ nghệ (Iron Triangle)



Cố định 2 trong 3 yếu tố $\Rightarrow$ yếu tố còn lại buộc phải thay đổi

Hình 18.1 - Tam giác đánh đổi kỹ nghệ (Iron Triangle): Phạm vi Thời gian Chi phí, với Chất lượng ở trung tâm.

Bài tập Thực hành

Bài 1. Áp dụng tam giác đánh đổi (Hình 18.1) cho một dự án thực tế bạn từng tham gia: khi deadline bị rút ngắn, đội của bạn đã đánh đổi ở góc nào? Hậu quả là gì?

Bài 2. Viết một tình huống giả định (3-4 câu) mà một kỹ sư phần mềm phải cân nhắc giữa lợi ích kỹ thuật thuần túy và trách nhiệm đạo đức/an toàn công cộng.

Bài 3. So sánh cách một kỹ sư cơ khí và một kỹ sư phần mềm cùng áp dụng nguyên lý 'tối ưu hóa dưới ràng buộc' - nêu 1 điểm giống và 1 điểm khác biệt đặc thù của phần mềm.